

Rapport d'Alternance

Ingénieur troisième année

Développement d'un logiciel de documentation des données géographiques



Marcel Assie

Soutenu le 01/09/2026

Organisme d'accueil

ISOGEO
26 Rue du Faubourg Saint-Antoine, 75012 Paris
Equipe Produit

Maître(s) d'apprentissage

Simon SAMPERE

Professeur(s) référent(s)

Laurent BRETON

Rapporteur principal

Période d'alternance : Du 08/09/2025 au 07/09/2026

Nombre de pages : XX

Version du document : 1.0

Remerciements

Je tiens à exprimer mes sincères remerciements à toutes les personnes qui ont contribué, de près ou de loin, au bon déroulement de mon alternance et à la réalisation de ce rapport.

Je remercie tout d'abord Monsieur Mathieu BECKER, directeur d'ISOGEO, de m'avoir accueilli au sein de l'entreprise dans le cadre de ma troisième année de formation d'ingénieur à Géodata Paris. Cette expérience m'a permis d'évoluer dans un environnement professionnel stimulant et de participer directement à l'évolution d'un produit consacré au recensement, à la documentation et à la valorisation des données géographiques.

J'adresse également mes remerciements à Madame Laurence DELAVALD, responsable administrative d'ISOGEO, pour son accueil, sa disponibilité et son accompagnement dans les différentes démarches administratives liées à mon alternance.

Je remercie chaleureusement mon tuteur d'alternance, Simon SAMPERE, pour la confiance qu'il m'a accordée, sa disponibilité et la qualité de son accompagnement tout au long de cette année. Ses conseils techniques et méthodologiques, ainsi que ses retours réguliers, m'ont permis de mieux comprendre les enjeux du produit Scan et d'améliorer ma manière d'analyser, de concevoir et de valider une solution logicielle.

Mes remerciements s'adressent également à l'ensemble des membres de l'équipe Produit d'ISOGEO, notamment Léa, Julie, Léo et Philippe, pour leur accueil, leur disponibilité et les nombreux échanges constructifs qui ont accompagné mon travail. Leur connaissance du produit ainsi que toutes les solutions externes liées au produit ont été essentielles pour comprendre l'existant et proposer des développements cohérents avec l'architecture de Scan.

Je remercie également toutes les personnes ayant participé aux échanges techniques et méthodologiques, aux revues de code et aux différentes phases de validation. Leurs remarques ont contribué à améliorer la robustesse des traitements développés et la qualité générale de la solution.

Je souhaite enfin remercier mon professeur référent, Laurent BRETON, pour son suivi et ses conseils durant cette période d'alternance, ainsi que l'ensemble de l'équipe pédagogique de Géodata Paris pour les connaissances et les méthodes transmises au cours de ma formation d'ingénieur en géomatique.

Enfin, je remercie toutes les personnes qui m'ont soutenu et encouragé tout au long de cette alternance et pendant la rédaction de ce rapport.

Résumé

Le produit Scan d'ISOGEO permet d'explorer différentes sources de données géographiques afin d'identifier leur contenu et d'en extraire les métadonnées. Ces informations alimentent ensuite le processus de catalogage, qui constitue le cœur de l'activité d'ISOGEO en facilitant le recensement, la documentation et la valorisation des données géographiques. Scan prend notamment en charge des bases de données, des géodatabases d'entreprise ainsi que plusieurs formats de fichiers géographiques.

Historiquement, l'ensemble de ces traitements reposait sur des workbenches FME, ce qui entraînait une dépendance externe et des coûts de licence. ISOGEO a donc engagé leur remplacement progressif par des scripts Python intégrés directement au produit. Ma mission a consisté à poursuivre cette migration, déjà initiée pour certaines sources comme PostGIS et les géodatabases d'entreprise Esri.

Après une phase d'analyse et de documentation de l'existant, j'ai participé au développement de traitements pour plusieurs sources, notamment les données vectorielles et raster, GeoPackage, les données CAO, Oracle, SQL Server et GeoParquet. Les travaux les plus récents ont également porté sur les nuages de points aux formats LAS et LAZ. Ces scripts permettent d'extraire les tables, les champs, les projections, les géométries, les emprises spatiales et les autres métadonnées attendues par Scan, ainsi que de générer une signature des données.

La bibliothèque GDAL/OGR a été principalement utilisée, complétée par des bibliothèques adaptées à certains formats spécifiques, comme LibreDWG ou encore SQLAlchemy. Les scripts ont ensuite été organisés dans une architecture modulaire et préparés pour être distribués sous la forme d'un exécutable Windows autonome avec PyInstaller. Cet exécutable est généré dans un environnement Docker afin de rendre le processus reproductible et indépendant des bibliothèques installées sur le poste du développeur. Des tests de régression et des tests d'intégrité de la signature permettent également de valider les traitements.

Ces travaux contribuent à réduire la dépendance de Scan à FME, à faciliter la maintenance des traitements et à renforcer leur intégration dans l'environnement logiciel d'ISOGEO.

Mots-clés : géomatique ; Scan ; ISOGEO ; Python ; migration FME ; métadonnées ; signature ; GDAL/OGR ; GeoPackage ; GeoParquet ; CAO ; bases de données spatiales ; nuages de points ; Docker ; PyInstaller.

Abstract

ISOGEO's Scan product explores various geospatial data sources to identify their contents and extract their metadata. This information supports the data cataloguing process, which lies at the core of ISOGEO's activities by facilitating the inventory, documentation and enhancement of geospatial data. Scan supports databases, enterprise geodatabases and several geospatial file formats.

Historically, all of these processes relied on FME workbenches, resulting in an external dependency and additional licensing costs. ISOGEO therefore began progressively replacing them with Python scripts integrated directly into the product. My work-study assignment consisted in continuing this migration, which had already been initiated for sources such as PostGIS and Esri enterprise geodatabases.

After analysing and documenting the existing developments, I contributed to the implementation of processing components for several sources, including vector and raster data, GeoPackage, CAD data, Oracle, SQL Server and GeoParquet. The most recent work also focused on point cloud data in LAS and LAZ formats. These scripts extract the tables, fields, coordinate reference systems, geometries, spatial extents and other metadata required by Scan. They also generate data signatures.

GDAL/OGR was the main library used, together with additional libraries adapted to specific formats, such as LibreDWG or even SQLAlchemy. The scripts were then organised within a modular architecture and prepared for distribution as a standalone Windows executable using PyInstaller. This executable is built in a Docker environment to ensure a reproducible process independent of the libraries installed on the developer's computer. Regression tests and signature integrity tests are also used to validate the processing components.

This work contributes to reducing Scan's dependency on FME, facilitating the maintenance of its processing components and strengthening their integration into ISOGEO's software environment.

Keywords : geomatics ; Scan ; ISOGEO ; Python ; FME migration ; metadata ; data signature ; GDAL/OGR ; GeoPackage ; GeoParquet ; CAD ; spatial databases ; point clouds ; Docker ; PyInstaller.

Glossaire et liste des sigles

Liste des sigles

Sigle	Signification
API	<i>Application Programming Interface</i> . Interface permettant à plusieurs composants logiciels ou applications de communiquer entre eux.
CAO	Conception assistée par ordinateur. Domaine regroupant les outils et formats de dessin technique, comme les fichiers DWG ou DXF.
CRS	<i>Coordinate Reference System</i> . Système de référence de coordonnées utilisé pour localiser des objets géographiques.
DGN	Format de fichier CAO notamment associé à certains outils de dessin technique.
DWG	Format de fichier CAO couramment utilisé par les logiciels de dessin assisté par ordinateur.
DXF	<i>Drawing Exchange Format</i> . Format CAO utilisé pour échanger des dessins entre logiciels.
EPSG	Référentiel de codes permettant d'identifier les systèmes de coordonnées et les transformations géodésiques.
FME	<i>Feature Manipulation Engine</i> . Plateforme d'intégration et de transformation de données développée par Safe Software.
GDAL	<i>Geospatial Data Abstraction Library</i> . Bibliothèque libre permettant de lire, écrire et transformer de nombreux formats de données géospatiales.
LAS	Format de fichier utilisé pour stocker des nuages de points, notamment issus de relevés LiDAR.
LAZ	Version compressée du format LAS, utilisée pour réduire le volume de stockage des nuages de points.
OGR	Composant de GDAL principalement consacré à la lecture, à l'écriture et au traitement des données vectorielles.
PDAL	<i>Point Data Abstraction Library</i> . Bibliothèque spécialisée dans la lecture et le traitement des nuages de points.
PR	<i>Pull Request</i> . Demande d'intégration de modifications dans un projet versionné, généralement relue avant d'être acceptée.
RSE	Responsabilité sociétale des entreprises.
SDF	Format de données géographiques ou CAO selon les contextes, étudié puis écarté du périmètre initial de développement.

Sigle	Signification
SGBD	Système de gestion de base de données.
SIG	Système d'information géographique.
SQL	<i>Structured Query Language</i> . Langage utilisé pour interroger et manipuler des bases de données relationnelles.

Glossaire

Terme	Définition
ArcPy	Bibliothèque Python fournie par Esri permettant d'automatiser des traitements réalisés avec ArcGIS et d'accéder aux propriétés des données géographiques Esri.
Base de données spatiale	Base de données capable de stocker, d'interroger et de manipuler des objets géographiques tels que des points, des lignes et des polygones.
Conda	Gestionnaire d'environnements et de dépendances Python. Dans le projet, il a été retenu pour construire un environnement de packaging plus stable autour de GDAL et PyInstaller.
Donnée raster	Donnée géographique organisée sous la forme d'une grille de cellules ou de pixels, utilisée notamment pour représenter des images aériennes, des modèles numériques ou des cartes continues.
Donnée vectorielle	Donnée géographique représentée par des géométries telles que des points, des lignes ou des polygones, auxquelles peuvent être associés des attributs.
Documentation des métadonnées	Étape consistant à extraire les informations décrivant une ressource : nom, format, projection, géométrie, attributs, emprise ou encore dates disponibles. Dans Scan, cette étape est aussi appelée <i>lookup</i> .
Docker	Outil permettant d'exécuter une application dans un environnement isolé et reproductible. Il a été utilisé pour générer l'exécutable Windows dans des conditions maîtrisées.
Driver	Composant logiciel permettant à GDAL/OGR de lire ou d'écrire un format de fichier ou une source de données particulière.
Emprise spatiale	Rectangle minimal décrivant les limites géographiques d'un jeu de données. Elle est généralement définie par ses coordonnées minimales et maximales.

Terme	Définition
Enveloppe convexe	Plus petit polygone convexe contenant l'ensemble des géométries d'un jeu de données.
GeoPackage	Format de fichier géographique pouvant contenir plusieurs couches dans un même fichier, par exemple des données vectorielles, des rasters ou des tables.
GeoJSON	Format d'échange de données géographiques textuel, couramment utilisé pour représenter des objets vectoriels et leurs attributs.
GeoParquet	Format de stockage géographique reposant sur Parquet. Il permet de stocker des données vectorielles de manière efficace, notamment pour des usages d'analyse.
Géodatabase	Structure de stockage de données géographiques utilisée notamment par les solutions Esri. Elle peut être locale ou hébergée dans un système de gestion de base de données.
Géomatique	Discipline regroupant les méthodes et technologies utilisées pour acquérir, stocker, traiter, analyser et représenter des données localisées.
Métadonnée	Information décrivant une ressource de données, par exemple son nom, sa structure, son format, sa projection, son emprise ou sa date de modification.
Laspy	Bibliothèque Python utilisée pour lire et écrire des fichiers de nuages de points aux formats LAS et LAZ.
LibreDWG	Bibliothèque libre spécialisée dans la lecture des fichiers DWG. Elle a été étudiée pour accéder au contenu des données CAO dans le cadre de Scan.
Listing	Étape de Scan consistant à inventorier les ressources accessibles dans une source de données avant de les documenter ou de les signer.
Lookup	Nom utilisé dans la documentation technique de Scan pour désigner l'étape de documentation. Elle consiste à extraire les métadonnées décrivant une ressource.
MkDocs	Outil permettant de générer un site statique de documentation. Il a été utilisé avec <i>mkdocstrings</i> pour produire une documentation à partir du code Python.
Nuage de points	Ensemble de points décrivant une surface ou un objet en trois dimensions. Ces données peuvent être très volumineuses et sont souvent stockées dans des fichiers LAS ou LAZ.
Oracle Spatial	Extension spatiale d'Oracle permettant de stocker et d'interroger des données géographiques dans une base Oracle.
Packaging	Processus consistant à regrouper une application et ses dépendances afin de permettre son installation ou son exécution dans un environnement cible.

Terme	Définition
PostGIS	Extension spatiale du système de gestion de base de données PostgreSQL. Elle permet de stocker et d'interroger des géométries ainsi que d'effectuer des traitements spatiaux.
PyInstaller	Outil permettant de transformer une application Python et ses dépendances en un exécutable pouvant être lancé sans installation préalable de Python.
Scan	Produit d'ISOGEO chargé d'explorer des sources de données, d'identifier les ressources qu'elles contiennent et d'extraire les métadonnées nécessaires à leur documentation dans l'écosystème ISOGEO.
Signature	Empreinte calculée à partir d'une ressource afin de vérifier son intégrité et de détecter une éventuelle évolution de son contenu.
Shapefile	Format de fichier vectoriel Esri très utilisé pour échanger des données géographiques.
Source de données	Emplacement ou système à partir duquel Scan analyse des ressources. Une source peut notamment correspondre à une base de données, une géodatabase, un dossier ou un fichier géographique.
SQL Server	Système de gestion de base de données relationnelle de Microsoft, pouvant aussi stocker des données spatiales.
Test de régression	Test visant à vérifier qu'une modification du logiciel n'altère pas un comportement précédemment validé. Dans ce projet, les résultats Python sont comparés aux résultats produits par les traitements FME de référence.
Workbench FME	Fichier de travail FME décrivant graphiquement une chaîne de lecture, de transformation et d'écriture de données.

Table des matières

Glossaire et liste des sigles	i
Liste des sigles.....	i
Glossaire.....	ii
Liste des figures	viii
Liste des tableaux.....	ix
1 Contexte et objectifs de l’alternance.....	1
1.1 Introduction.....	1
1.2 Présentation du contexte général	2
1.2.1 Présentation de l’organisme d’accueil.....	2
1.2.2 Environnement de travail	4
1.2.3 Approche RSE de l’organisme d’accueil	5
1.3 Présentation du sujet d’alternance	6
1.3.1 Contexte technique du produit Scan.....	6
1.3.2 Migration des traitements FME	7
1.3.3 Problématique et objectifs de la mission	9
2 Prise en main et conception de la solution.....	10
2.1 Découverte de l’environnement Isogeo	10
2.1.1 Formation aux solutions existantes.....	10
2.1.2 Prise en main du produit Scan.....	10
2.2 Analyse et documentation de l’existant	11
2.2.1 Étude du code Python existant	11
2.2.2 Documentation de l’existant	11
2.3 Expérimentations avec GDAL et OGR	12
2.3.1 Prise en main des bibliothèques	12
2.3.2 Réalisation de tests techniques	13
2.4 Conception de la solution.....	13
2.4.1 Définition de l’architecture.....	13
2.4.2 Définition de la méthode de validation	13
2.5 Préparation du packaging.....	14
2.5.1 Objectifs du packaging.....	14
2.5.2 Mise en place d’un environnement Docker	15
3 Développement des scripts Python	17
3.1 Organisation générale des développements.....	17
3.1.1 Une progression par formats de données	17

3.1.2	Périmètre fonctionnel selon les formats	17
3.2	Développements sur les fichiers géographiques	18
3.2.1	Données rasters et vectorielles	18
3.2.2	GeoPackage	19
3.2.3	GeoParquet.....	19
3.2.4	Données CAO.....	20
3.3	Développements sur les bases de données	21
3.3.1	Oracle	21
3.3.2	SQL Server.....	22
3.4	Traitement des nuages de points	23
3.5	Gestion des erreurs et des connexions	24
3.5.1	Gestion des erreurs	24
3.5.2	Modes de connexion aux bases de données	24
3.6	Principe de signature des données	25
3.6.1	Signature des fichiers GeoPackage.....	25
3.6.2	Signature des bases de données	25
3.7	Intégration, revue de code et exécutable	26
3.7.1	Revue de code et intégration progressive	26
3.7.2	Suivi de l'exécutable au fil des développements	26
4	Résultats, validation et bilan.....	28
4.1	Résultats obtenus	28
4.1.1	Résultats du listing	28
4.1.2	Résultats du lookup	29
4.1.3	Résultats de la signature.....	32
4.2	Validation des traitements développés	32
4.2.1	Tests de non-régression	33
4.2.2	Tests d'intégrité de la signature	35
4.2.3	Tests de performance	37
4.3	Limites rencontrées	37
4.3.1	Limites liées aux formats	37
4.3.2	Limites liées aux environnements de test	38
4.3.3	Limites liées au packaging	38
4.4	Apports de l'alternance	38
4.4.1	Apports techniques	38
4.4.2	Apports méthodologiques	39
4.4.3	Apports professionnels	39
4.5	Perspectives et recommandations.....	39
4.5.1	Poursuite de la migration.....	40

4.5.2	Enrichissement des métadonnées	40
4.6	Conclusion.....	40

Table des figures

1.1	Organisation de l'entreprise Isogeo en novembre 2025	3
2.1	Processus de documentation du code source	12
2.2	Passage du code brut au site statique de documentation	12
4.1	Exemple de résultat de listing pour un fichier GeoPackage	29
4.2	Exemple de résultat de listing pour une base de données SQL Server	29
4.3	Exemple de résultat de lookup pour un fichier Shapefile	30
4.4	Exemple de résultat de lookup pour une donnée CAO au format DWG	31
4.5	Exemples de résultats de signature selon le type de donnée	32
4.6	Exemples de tests de non-régression	34
4.7	Méthode de décision utilisée pour valider les tests de non-régression	35
4.8	Exemple de résultats des tests d'intégrité de la signature	36
4.9	Exemples de résultats de tests de performance	37

Liste des tableaux

2.1	Principaux tests prévus pour valider les traitements Python	14
3.1	Progression des développements selon les formats étudiés	18
4.1	Synthèse des résultats selon les traitements de Scan	28
4.2	Principales familles de tests utilisées pour valider les traitements	33

CHAPITRE 1

Contexte et objectifs de l’alternance

1.1 Introduction

Les données géographiques occupent une place importante dans le fonctionnement de nombreuses organisations. Elles sont utilisées dans des domaines variés tels que l’aménagement du territoire, la gestion des réseaux, l’environnement, les transports ou encore l’urbanisme. Cependant, ces données sont souvent réparties entre plusieurs bases de données, serveurs, répertoires de fichiers et applications métiers. Cette dispersion peut rendre leur identification, leur compréhension et leur réutilisation difficiles.

Isogeo développe des solutions permettant aux organisations de mieux recenser, documenter et valoriser leur patrimoine de données géographiques. Parmi ces solutions, le produit Scan permet d’explorer différentes sources de données, d’identifier les ressources qu’elles contiennent et d’en extraire les informations techniques nécessaires à leur catalogage. Il peut notamment analyser des bases PostgreSQL/PostGIS, Oracle et Microsoft SQL Server, des géodatabases d’entreprise Esri ainsi que plusieurs formats de fichiers géographiques.

Historiquement, l’ensemble des traitements réalisés par Scan reposait sur des workbenches FME. Ces traitements permettaient de parcourir les différentes sources, d’analyser les ressources disponibles et de produire les informations attendues par le produit. Toutefois, cette organisation rendait le fonctionnement de Scan dépendant de l’installation et de la licence d’un logiciel externe.

Dans une démarche d’évolution du produit, Isogeo a donc engagé la migration progressive de l’ensemble de ces traitements FME vers une solution développée en Python. L’objectif est de rendre Scan plus autonome, de faciliter la maintenance de ses traitements et de permettre leur exécution sous la forme d’un programme Windows ne nécessitant pas l’installation préalable de Python ou de FME.

Mon alternance s’inscrit dans ce contexte. Ma mission principale consiste à poursuivre cette migration en analysant les traitements existants, leurs entrées, leurs sorties et les règles métier qu’ils appliquent, puis en développant des scripts Python capables de produire des résultats équivalents. Elle comprend également la prise en charge de différentes sources de données, la gestion des erreurs, la mise en place de tests comparatifs et la préparation d’un exécutable Windows autonome.

La problématique générale de ce travail peut ainsi être formulée de la manière suivante :

Comment remplacer intégralement les traitements FME utilisés par Scan par

une solution Python autonome, tout en garantissant la fiabilité des résultats et la compatibilité avec les différentes sources de données prises en charge ?

Pour répondre à cette problématique, le travail a été organisé autour de l'analyse de l'existant, du développement des nouveaux traitements, de leur intégration dans l'architecture de Scan et de leur validation par comparaison avec les résultats FME de référence.

Ce rapport est structuré en quatre chapitres. Le premier présente le contexte de l'alternance, l'entreprise Isogeo, le produit Scan ainsi que les objectifs de la mission. Le deuxième expose l'analyse de l'existant et les choix techniques retenus pour concevoir la solution. Le troisième décrit les développements réalisés, la préparation de l'exécutable et la démarche de validation. Enfin, le quatrième présente les résultats obtenus, les apports de l'alternance, les limites rencontrées et les perspectives du projet.

1.2 Présentation du contexte général

1.2.1 Présentation de l'organisme d'accueil

Isogeo est une entreprise française spécialisée dans le recensement, la documentation et la valorisation des données géographiques. Elle développe une solution de catalogage destinée aux organisations qui produisent, administrent ou utilisent un volume important de données géographiques.

Son activité répond à une difficulté fréquemment rencontrée au sein des systèmes d'information : les données sont souvent réparties entre plusieurs bases de données, serveurs, répertoires de fichiers et applications métiers. Cette dispersion peut rendre difficile l'identification des ressources disponibles, la compréhension de leur contenu, l'évaluation de leur qualité et leur réutilisation par les différents utilisateurs.

La solution développée par Isogeo permet d'automatiser la constitution et la mise à jour d'un catalogue de données géographiques. Elle aide les organisations à disposer d'un inventaire centralisé de leur patrimoine de données, à enrichir les fiches de métadonnées associées et à faciliter leur recherche, leur consultation et leur diffusion.

Les utilisateurs de la solution appartiennent aussi bien au secteur public qu'au secteur privé. Il peut notamment s'agir de collectivités territoriales, de bureaux d'études ou de grandes organisations disposant d'un patrimoine important de données géographiques.

Organisation de l'entreprise

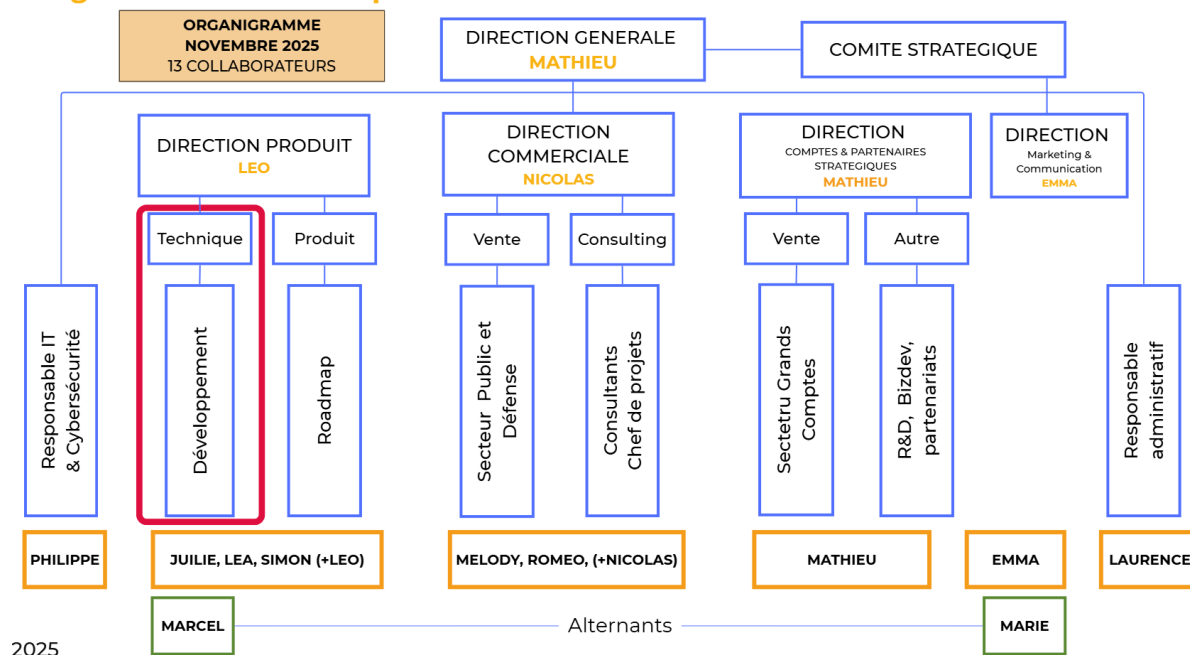


FIGURE 1.1 : Organisation de l'entreprise Isogeo en novembre 2025

Produits et service

L'écosystème Isogeo s'organise autour de plusieurs services complémentaires, chacun associé à un usage précis dans le cycle de vie des catalogues de données. L'application principale, appelée *App*, constitue l'espace de travail des utilisateurs. Elle sert à recenser les jeux de données, à documenter les fiches de métadonnées, à les enrichir et à valoriser les catalogues auprès des publics concernés. Le service *Scan* intervient plus en amont : il permet de repérer automatiquement les données présentes dans les fichiers, les bases de données ou les serveurs cartographiques afin de préparer ou mettre à jour les fiches associées.

La partie *Manage* est destinée à l'administration de la solution. Elle permet de gérer les groupes de travail, les utilisateurs, les applications, les invitations ainsi que les listes de mots-clés et de thématiques utilisées pour structurer les catalogues. D'autres services sont tournés vers la diffusion et l'accès aux données. L'*Extracteur* permet de télécharger des données dans différents formats et projections, tandis qu'*OpenCatalog* sert à publier un catalogue sur le web et à le rendre consultable depuis un site ou un portail. Le *Portail de données* propose une interface de diffusion et de valorisation plus complète, avec des fonctionnalités de recherche, de suivi et d'animation autour des catalogues publiés. Enfin, les plugins pour QGIS et ArcGIS Pro permettent aux utilisateurs de retrouver, consulter et ajouter des données directement depuis leur logiciel SIG habituel, à partir des métadonnées documentées dans Isogeo.

1.2.2 Environnement de travail

L’alternance s’est déroulée au sein de l’équipe Produit d’Isogeo. Cette équipe participe à la conception, au développement, à la maintenance et à l’amélioration continue des différents composants de la solution.

Mon travail a été réalisé sous la responsabilité de Simon SAMPERE , maître d’apprentissage, et en collaboration avec les membres de l’équipe impliqués dans le développement et la maintenance du produit Scan.

Organisation du travail

Durant les premiers mois, un point de suivi quotidien était organisé avec mon tuteur d’alternance. Il permettait de revenir sur les tâches réalisées la veille, de définir les priorités de la journée et de suivre mon intégration au sein de l’entreprise.

Le travail s’inscrivait aussi dans une organisation agile. Chaque matin, l’équipe participait à un *stand-up* d’environ quinze minutes. Chacun y présentait ce qu’il avait fait, ce qu’il prévoyait de faire et les éventuels points de blocage.

D’autres réunions rythmaient les sprints. Les *sprint reviews* permettaient de présenter les travaux réalisés. Les *sprint retrospectives* servaient à identifier les points d’amélioration dans l’organisation de l’équipe. Les *planning meetings* permettaient enfin de préparer les tâches du sprint suivant.

Le suivi reposait également sur un système de tickets. Chaque ticket décrivait une fonctionnalité, une anomalie ou une amélioration attendue. Cette organisation facilitait la priorisation des travaux et permettait de conserver une trace des décisions prises.

Outils utilisés

La réalisation de la mission a nécessité l’utilisation de plusieurs outils :

- des IDE pour écrire, exécuter et déboguer les scripts Python ;
- Git pour la gestion des versions et le suivi des modifications ;
- FME pour consulter et exécuter les workbenches servant de référence ;
- GDAL et OGR pour traiter certains formats de données géographiques ;
- ArcGIS Pro et ArcPy pour réaliser des tests propres à l’environnement Esri ;
- des bases de données hébergées sur des serveurs pour tester les scripts ;
- Docker et des machines virtuelles pour reproduire les environnements d’exécution et de test ;

Collaboration et contrôle de la qualité

La collaboration au sein de l'équipe reposait sur des échanges techniques, des revues de code et différentes phases de test. Les modifications étaient enregistrées dans le gestionnaire de versions afin de suivre leur évolution, de les comparer et de les intégrer progressivement au projet.

Une attention particulière était portée à la lisibilité du code, à la gestion des erreurs et à la stabilité des échanges entre les différents composants. Les scripts développés devaient en effet s'intégrer dans une chaîne de traitement existante et respecter des formats d'entrée et de sortie déjà utilisés par les autres parties de Scan.

La documentation constituait également un élément important du travail. Elle permettait de présenter le comportement des traitements, leurs dépendances, leurs paramètres et les éventuelles limites identifiées.

1.2.3 Approche RSE de l'organisme d'accueil

La responsabilité sociétale des entreprises, généralement désignée par le sigle RSE, correspond à la prise en compte des enjeux environnementaux, sociaux et économiques dans les activités et les décisions d'une organisation.

La présente partie ne constitue pas un audit de la démarche RSE d'Isogeo. Elle vise à mettre en évidence les liens pouvant être établis entre les activités de l'entreprise, son organisation et certains principes associés à la responsabilité sociétale.

Gouvernance et valorisation des données

La principale contribution de la solution Isogeo concerne la gouvernance et la valorisation du patrimoine de données des organisations. La mise en place d'un catalogue permet de mieux identifier les ressources existantes, leurs producteurs, leurs conditions d'utilisation et leur niveau de documentation.

Cette connaissance favorise la réutilisation des données disponibles et limite le risque de produire plusieurs fois une information déjà existante. Elle contribue également à améliorer la traçabilité des données et à fiabiliser les processus métiers qui en dépendent.

Pour les organismes publics, une meilleure documentation facilite également la diffusion et l'ouverture des données. Elle peut ainsi contribuer à renforcer la transparence de l'action publique et l'accès aux informations produites par les administrations.

Enjeux environnementaux

Les activités numériques ont un impact environnemental lié notamment à l'utilisation des infrastructures informatiques, au stockage des données et à la consommation de ressources de calcul. Une gestion plus rigoureuse des données peut contribuer indirectement à limiter certains traitements inutiles ou redondants.

Le fonctionnement de Scan s'inscrit dans cette logique. Une signature est calculée à partir du contenu des données afin de détecter leur modification. Lorsqu'une ressource n'a pas changé depuis l'exécution précédente, il n'est pas nécessaire de réaliser de nouveau l'ensemble de son traitement. Ce mécanisme permet d'optimiser les temps d'exécution et d'éviter certaines opérations inutiles.

La migration de FME vers une solution Python autonome permet également de simplifier les prérequis nécessaires au fonctionnement du produit. Aucun gain environnemental précis ne peut toutefois être affirmé sans une mesure comparative des ressources consommées avant et après la migration.

Transmission des connaissances et qualité du travail

La documentation et le partage des connaissances occupent une place importante dans le fonctionnement de l'équipe. Les revues de code, les échanges techniques et la rédaction de documentation rendent les développements plus compréhensibles et limitent la dépendance à une seule personne.

L'accueil d'étudiants en stage ou en alternance participe également à la transmission des compétences dans les domaines du développement logiciel, de la géomatique et de la gouvernance des données.

Enfin, les solutions développées par Isogeo rendent les données géographiques plus facilement accessibles à leurs utilisateurs. En facilitant leur recherche, leur compréhension et leur réutilisation, elles contribuent à réduire certains obstacles techniques liés à leur exploitation.

1.3 Présentation du sujet d'alternance

1.3.1 Contexte technique du produit Scan

Scan est le composant de la solution Isogeo chargé d'explorer les sources de données configurées par les utilisateurs. Il identifie les ressources présentes, en extrait les informations techniques, puis transmet les résultats à la plateforme Isogeo afin de créer ou de mettre à jour les fiches de métadonnées correspondantes.

Le service installé dans l'environnement du client est appelé *Isogeo Worker*. Il s'exécute sur une machine Windows disposant des droits nécessaires pour accéder aux sources ciblées. Les opérations réalisées par Scan sont des opérations de lecture et n'ont pas pour objectif de modifier les données des clients.

Le traitement d'une source repose principalement sur trois opérations : le listing, la signature et l'extraction des métadonnées.

Le listing

Le listing consiste à inventorier les ressources accessibles à partir d'un point d'entrée. Selon le type de source, une ressource peut correspondre à une table, une vue, une couche géographique, un fichier ou un jeu de données.

Cette étape permet à Scan de connaître la liste des éléments disponibles avant de déterminer ceux qui doivent être analysés plus précisément.

La signature

La signature correspond à une empreinte calculée à partir du contenu d'une ressource. Elle permet à Scan de reconnaître une donnée déjà rencontrée et de détecter une éventuelle modification entre deux exécutions.

Cette information joue un rôle important dans la mise à jour du catalogue. Si une donnée n'a pas changé, certains traitements peuvent être évités. Si sa signature évolue, Scan peut déclencher une nouvelle extraction de ses métadonnées.

La documentation

L'étape de documentation des données, également appelée *lookup* dans la documentation technique de Scan, produit les informations décrivant la ressource analysée. Elle peut par exemple récupérer son nom, son emplacement, son format, son système de coordonnées ou encore le nombre d'entités qu'elle contient.

Selon le type de donnée, Scan peut aussi extraire des informations plus spécifiques, comme le type de géométrie, l'emprise spatiale ou certaines propriétés propres aux données raster. Ces éléments servent ensuite à compléter la fiche de métadonnées associée.

Les résultats doivent être structurés de manière à pouvoir être interprétés par les autres composants de Scan. La nouvelle implémentation Python doit donc respecter le contrat déjà utilisé par le produit, notamment la structure des résultats, les paramètres d'exécution et les codes d'erreur.

Diversité des sources prises en charge

Scan doit fonctionner dans des environnements techniques variés. Les données peuvent être stockées dans des bases de données, des géodatabases Esri, des fichiers vectoriels ou raster, ou encore être accessibles depuis certains services géographiques.

Cette diversité rend le produit plus complexe à maintenir. Chaque source a ses propres règles de connexion, ses formats et ses comportements. Une même opération peut donc nécessiter un traitement différent selon la technologie utilisée.

1.3.2 Migration des traitements FME

FME est une plateforme spécialisée dans l'intégration et la transformation de données. Elle propose de nombreux lecteurs et composants adaptés aux données géographiques. Pour cette raison, l'ensemble des traitements historiques de Scan reposait sur des workbenches FME.

Le choix d'engager une migration ne remet pas en cause les capacités fonctionnelles de FME. Il répond principalement à un besoin d'autonomie et d'évolution du produit.

Dépendance à un logiciel externe

L'exécution des workbenches nécessite une installation compatible de FME ainsi qu'une licence disponible. Le fonctionnement de Scan dépend donc d'un logiciel tiers qui doit être installé, configuré et maintenu dans l'environnement du client.

Cette dépendance augmente le nombre de prérequis nécessaires au déploiement du produit et entraîne des coûts de licence. Elle peut également compliquer les opérations de support lorsqu'un problème dépend de la version de FME, de l'activation de la licence ou d'un composant particulier de l'installation.

Maintenance des traitements

Les workbenches FME offrent une représentation graphique des traitements, mais leur maintenance peut devenir complexe lorsque le nombre de sources et de cas particuliers augmente.

Une implémentation Python permet de regrouper les comportements communs dans des classes et des fonctions réutilisables. Elle facilite également la gestion des versions, les revues de code, la mise en place de tests automatisés et l'intégration d'outils de contrôle de la qualité.

Intégration dans l'architecture de Scan

Le remplacement des workbenches par des composants Python permet de rapprocher les traitements de données du reste du code de Scan.

Les scripts doivent également être distribués sous la forme d'un exécutable Windows autonome. La génération de cet exécutable repose sur PyInstaller. Elle est réalisée dans un environnement Docker afin de disposer d'un processus reproductible et indépendant des bibliothèques installées sur le poste du développeur.

Continuité fonctionnelle

Dans un premier temps, l'objectif de la migration est de conserver une isofonctionnalité avec les traitements FME existants. Les nouveaux traitements Python doivent donc produire des résultats compatibles avec ceux attendus par Scan et par les autres composants de la solution.

Au cours du travail, certains ajustements ont toutefois pu être réalisés. Ils permettaient de corriger des écarts, de simplifier certains traitements ou d'adapter le fonctionnement aux contraintes propres à chaque source de données.

La migration est réalisée progressivement, source par source. Avant le début de mon alternance, ce travail avait déjà été engagé pour certaines sources, notamment PostGIS et les géodatabases d'entreprise Esri.

1.3.3 Problématique et objectifs de la mission

La problématique générale de cette alternance peut être formulée de la manière suivante :

Comment remplacer intégralement les traitements FME utilisés par Scan par une solution Python autonome, tout en garantissant la fiabilité des résultats et la compatibilité avec les différentes sources de données prises en charge ?

Ma mission s'inscrit dans la poursuite de cette migration. Elle ne consistait pas à développer un nouveau produit indépendant, mais à faire évoluer un composant existant tout en maintenant sa compatibilité avec le reste de l'écosystème Isogeo.

Objectif principal

L'objectif principal était de concevoir, développer et valider des traitements Python capables de remplacer les traitements FME utilisés par Scan pour inventorier, signer et documenter différentes sources de données.

Les traitements développés devaient pouvoir être intégrés dans l'architecture existante et être exécutés de manière autonome dans un environnement Windows.

Objectifs spécifiques

Pour atteindre cet objectif principal, plusieurs objectifs spécifiques ont été définis :

- comprendre et documenter l'existant, comprendre l'architecture de Scan et les traitements FME servant de référence ;
- préparer un exécutable Windows autonome, généré dans un environnement Docker reproductible.
- développer en Python les opérations principales de Scan, notamment le listing, la signature et l'extraction des métadonnées ;
- comparer les résultats obtenus avec ceux de FME et mettre en place des tests de régression ;

Les principales sources étudiées au cours de la mission comprenaient notamment les formats de données vectorielles et raster, les données CAO (DWG, DXF), GeoPackage, Oracle, Microsoft SQL Server, GeoParquet et les nuages de points.

CHAPITRE 2

Prise en main et conception de la solution

2.1 Découverte de l'environnement Isogeo

2.1.1 Formation aux solutions existantes

Les premières semaines ont été consacrées à la découverte des solutions développées par Isogeo. Cette phase correspondait à la période d'intégration. Elle devait me permettre de comprendre l'entreprise, son organisation et le rôle de Scan dans l'ensemble de l'écosystème.

Au cours de cette période, j'ai regardé plusieurs webinaires de présentation de la solution. J'ai également lu des documents liés à l'entreprise, aux produits Isogeo et aux pratiques de développement mises en place au sein de l'équipe Produit.

Cette étape était importante pour mieux situer la mission dans le fonctionnement réel du produit.

Les formations ont porté sur les principaux services utilisés par les clients. L'*App* constitue l'espace de travail principal pour gérer les catalogues et documenter les fiches de métadonnées. *Manage* est destiné à l'administration des groupes, des utilisateurs et des applications associées. *OpenCatalog* et le *Portail de données* servent à publier et à valoriser les catalogues auprès des utilisateurs finaux.

D'autres outils complètent cet ensemble. L'*Extracteur* permet de rendre les données plus facilement téléchargeables. Les plugins QGIS et ArcGIS Pro facilitent l'accès aux données depuis les logiciels SIG utilisés au quotidien par les géomaticiens.

Cette découverte m'a permis de comprendre que Scan n'est pas un composant isolé. Il intervient en amont du catalogue. Son rôle est de recenser les ressources disponibles, d'en extraire les informations techniques utiles et de préparer la mise à jour des fiches de métadonnées.

2.1.2 Prise en main du produit Scan

La prise en main de Scan a ensuite permis d'identifier les principales étapes réalisées par le produit. Les traitements reposent sur trois opérations centrales : le listing, la signature et la documentation des données.

Le listing sert à inventorier les ressources accessibles dans une source. La signature permet de détecter si une ressource a changé depuis une exécution précédente. La documentation produit les informations techniques nécessaires à la création ou à la mise à jour d'une fiche de

métadonnées.

Cette phase a aussi permis de comprendre les contraintes du produit. Le logiciel doit fonctionner chez les clients, dans des environnements différents, avec des sources de données variées. Les traitements doivent donc être fiables, prévisibles et compatibles avec le fonctionnement existant.

La prise en main ne s'est pas limitée à une lecture théorique. Elle s'est faite progressivement, en échangeant avec l'équipe, en observant le comportement du produit et en réalisant des premiers tests sur des sources simples.

2.2 Analyse et documentation de l'existant

2.2.1 Étude du code Python existant

Avant mon arrivée, une partie de la migration avait déjà été engagée. Il existait donc du code Python à lire, à comprendre et à replacer dans l'architecture générale de Scan.

Cette étape a consisté à parcourir les modules existants, à identifier les classes principales et à comprendre la manière dont les traitements étaient organisés. L'objectif était de repérer les éléments déjà réutilisables et les choix de conception à conserver pour la suite du développement.

L'analyse du code a aussi permis de mieux comprendre les responsabilités de chaque module. Certains éléments étaient communs à plusieurs sources, comme la gestion des paramètres, la structure des résultats ou le traitement des erreurs. D'autres dépendaient directement du type de source analysée.

Cette lecture progressive a servi de base au travail de développement. Elle a évité de repartir de zéro et a permis de rester cohérent avec les choix déjà faits par l'équipe.

2.2.2 Documentation de l'existant

La documentation a accompagné cette phase d'analyse. Elle avait pour but de conserver les informations utiles à la poursuite de la migration et de rendre plus lisible le fonctionnement du code existant.

Il s'agissait principalement d'une documentation directement liée au code source. Les classes et les fonctions étaient décrites à l'aide de commentaires de documentation, placés sous leur définition. Ces descriptions précisaient le rôle du traitement, les paramètres attendus, les valeurs retournées et les cas particuliers à connaître.

Cette documentation était utile pendant la prise en main, mais aussi pour toute personne devant éventuellement reprendre le code source. Elle facilitait les échanges techniques et permettait de partager plus facilement ce qui avait été compris ou vérifié.

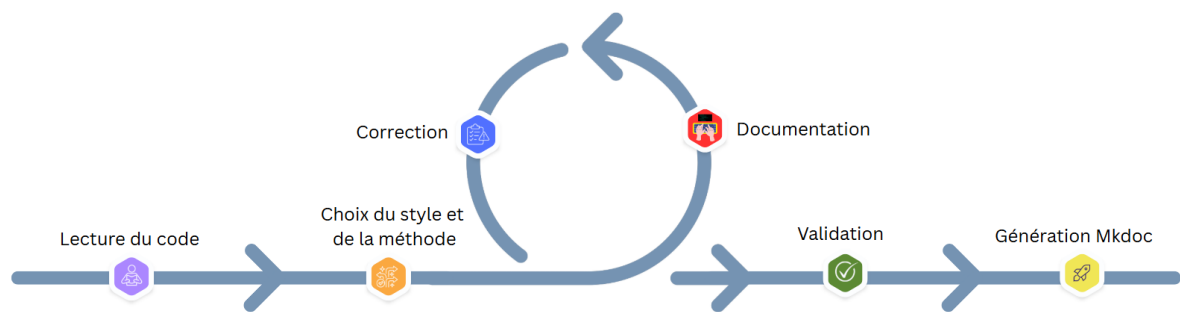


FIGURE 2.1 : Processus de documentation du code source

Afin de rendre cette documentation plus accessible, nous avons également produit un site statique avec *MkDocs* et *mkdocstrings*. Cet outil permet de générer automatiquement des pages de documentation à partir des informations présentes dans le code. Le résultat est plus simple à consulter qu'une lecture directe des fichiers Python.

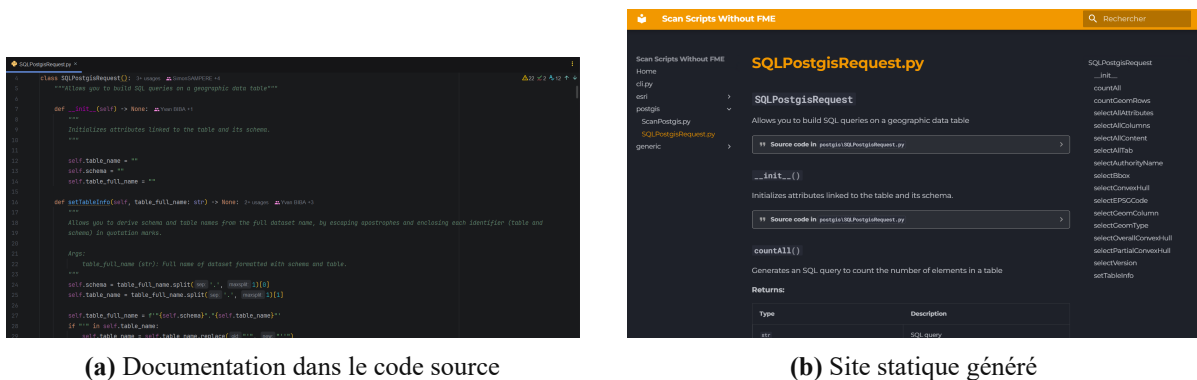


FIGURE 2.2 : Passage du code brut au site statique de documentation

2.3 Expérimentations avec GDAL et OGR

2.3.1 Prise en main des bibliothèques

GDAL et OGR sont des bibliothèques largement utilisées pour lire, manipuler et analyser des données géographiques. Dans le cadre de la mission, elles représentaient une solution importante pour remplacer une partie des traitements historiquement réalisés avec FME.

La prise en main a d'abord porté sur la lecture de fichiers vectoriels et raster. Il s'agissait de vérifier comment accéder aux couches, aux champs, au système de coordonnées, à l'emprise ou encore au nombre d'entités.

Ces premiers essais ont permis de mieux comprendre la manière dont GDAL et OGR présentent les données. Ils ont aussi montré que les informations disponibles peuvent varier selon les formats, les pilotes utilisés et la qualité des données en entrée.

2.3.2 Réalisation de tests techniques

Des tests techniques ont ensuite été réalisés sur les formats rasters et vecteurs. L'objectif était de vérifier ce que les bibliothèques permettaient d'obtenir et de repérer les comportements particuliers dès le début du travail.

Ces tests ont porté sur des opérations simples, comme l'ouverture d'une source, la lecture des couches, la récupération des champs ou l'accès aux informations géographiques principales. Ils ont aussi permis d'observer les erreurs renvoyées lorsque la donnée était absente, invalide ou mal structurée.

Cette étape a été utile pour préparer la suite du développement. Elle a permis d'identifier les possibilités offertes par GDAL et OGR, mais aussi certaines limites à prendre en compte dans l'architecture de la solution.

2.4 Conception de la solution

2.4.1 Définition de l'architecture

Après la phase de prise en main, il a été nécessaire de définir une organisation claire pour les nouveaux traitements. L'objectif était de construire une solution capable de prendre en charge plusieurs sources, sans dupliquer inutilement la logique commune.

L'architecture devait séparer les responsabilités. Les éléments communs devaient gérer les paramètres d'entrée, la structure des résultats, les erreurs et les étapes générales du traitement. Les éléments propres à chaque source devaient se concentrer sur les particularités du format ou de la technologie utilisée.

Cette organisation permettait de faciliter les évolutions futures. Lorsqu'une nouvelle source devait être prise en charge, l'idée était de pouvoir réutiliser le socle commun et d'ajouter uniquement les traitements spécifiques nécessaires.

La conception devait aussi tenir compte de l'intégration dans Scan. Les scripts ne devaient pas seulement fonctionner seuls. Ils devaient produire des résultats exploitables par le reste du produit et respecter les conventions déjà en place.

2.4.2 Définition de la méthode de validation

La méthode de validation dépendait du type de traitement étudié. Pour l'inventaire des données et la documentation, elle reposait principalement sur des tests de non-régression. L'objectif était de comparer les sorties produites par les nouveaux scripts Python avec les résultats obtenus par les workbenches FME de référence.

Pour la documentation des données, la comparaison portait sur l'ensemble des métadonnées

produites : nom de la ressource, système de projection, emprise, enveloppe convexe, champs, types de géométrie et autres informations attendues par Scan.

Pour la signature, la validation cherchait surtout à vérifier l'intégrité du calcul. Il fallait s'assurer que le hash restait stable lorsque la donnée ne changeait pas, mais qu'il évoluait lorsqu'une modification réelle était effectuée.

Type de test	Objectif	Application
Non-régression	Comparer les sorties Python avec les résultats FME de référence.	Inventaire et documentation.
Intégrité de la signature	Vérifier que le hash réagit correctement aux modifications de la donnée.	Signature.
Performance	Mesurer le comportement des scripts sur des volumes de données importants.	Inventaire, documentation et signature.

TABLE 2.1 : Principaux tests prévus pour valider les traitements Python

NB : les tests d'intégrité de la signature regroupaient plusieurs vérifications. La **constance** permettait de vérifier qu'une même donnée produisait la même signature entre deux exécutions. Les **cas limites** servaient à contrôler le comportement face à une table absente, une source invalide ou des types inconnus. L'**isolation** vérifiait qu'une modification sur une couche ne modifiait pas la signature d'une autre couche. Les tests **spatiaux** portaient sur les modifications géométriques ou d'emprise. Les tests **structurels** permettaient de détecter un changement de schéma, comme l'ajout d'une colonne. Enfin, les tests de **sensibilité** vérifiaient que toute modification significative de la donnée était bien prise en compte dans le hash.

Ces tests permettaient d'identifier rapidement les écarts avec le comportement historique de Scan. Lorsqu'une différence était détectée, elle était analysée pour déterminer s'il s'agissait d'une régression, d'un cas particulier ou d'un ajustement acceptable.

2.5 Préparation du packaging

2.5.1 Objectifs du packaging

Un autre enjeu important concernait la distribution des traitements Python. La solution devait pouvoir être exécutée dans un environnement Windows sans installation locale de Python.

Le packaging avait donc pour objectif de regrouper les scripts, les bibliothèques nécessaires et les dépendances géospatiales dans un exécutable autonome. Cette approche permettait de simplifier le déploiement et de limiter les prérequis côté utilisateur.

La difficulté principale venait des bibliothèques géospatiales. GDAL et OGR sont des outils puissants, mais leur installation peut être complexe dans un environnement Windows classique. Ils reposent sur plusieurs dépendances natives et leur configuration peut varier selon les versions installées.

Il était donc important de produire un exécutable qui embarque directement ces éléments. L'objectif était d'éviter aux utilisateurs et aux développeurs de se poser des questions sur l'installation manuelle des modules, en particulier pour GDAL. Le processus de génération devait donc être fiable, reproductible et maîtrisé.

2.5.2 Mise en place d'un environnement Docker

Pour répondre à ce besoin, la génération de l'exécutable devait être réalisée dans un environnement Docker. L'objectif était d'avoir Docker comme seul prérequis pour construire l'exécutable.

Cette organisation permettait de maîtriser l'environnement de construction. Les versions de Python, de PyInstaller, de GDAL et des autres dépendances pouvaient être fixées dans une image dédiée. La génération ne dépendait donc plus des bibliothèques installées sur le poste du développeur.

Plusieurs approches ont été étudiées pour intégrer GDAL dans cet environnement. Les premiers essais ont notamment porté sur les wheels Python, c'est-à-dire des paquets déjà préparés pour faciliter l'installation des bibliothèques. Cette solution était simple à tester, mais la source utilisée pour ces wheels n'était pas suffisamment stable ni maintenue de façon durable. Pour une solution comme Isogeo, qui doit rester fiable dans le temps, cette approche n'était donc pas la plus adaptée.

Une autre piste consistait à utiliser OSGeo, une distribution Windows regroupant plusieurs outils de géomatique, dont GDAL. Cette approche avait l'avantage d'être proche des installations classiques du domaine. En revanche, elle produisait un livrable beaucoup plus lourd et dépendait de chemins d'installation susceptibles de changer selon les versions. Cela risquait de rendre le Dockerfile plus difficile à maintenir.

L'approche Conda a finalement été retenue. Conda permet de créer un environnement isolé avec des versions précises des outils utilisés. Cela permettait de figer plus facilement les versions de Python, GDAL, PyInstaller et des bibliothèques associées. Cette solution produisait également un dossier final plus léger et un environnement plus stable.

Le processus a donc été organisé en deux étapes. Une première image Docker servait de base et contenait l'environnement Python avec les dépendances géospatiales principales. Une seconde image, dédiée à la construction, ajoutait le code source de Scan, installait les dépendances propres au projet et lançait le script de génération de l'exécutable.

Cette séparation facilitait la maintenance. L'image de base pouvait être réutilisée tant que les versions principales ne changeaient pas. L'image de construction pouvait ensuite être régénérée à partir du code courant.

Un point particulier concernait les données internes utilisées par GDAL, notamment celles de la bibliothèque PROJ. Elles devaient être correctement embarquées avec l'exécutable afin que l'application utilise les bonnes données au moment de son exécution, sans dépendre d'une installation présente sur la machine.

Cette étape préparait l'industrialisation de la solution. Elle devait permettre de passer progressivement de scripts de développement à un programme Windows généré de manière fiable, reproductible et utilisable dans le contexte réel de Scan.

CHAPITRE 3

Développement des scripts Python

3.1 Organisation générale des développements

3.1.1 Une progression par formats de données

Le développement des nouveaux traitements Python ne s'est pas fait en une seule fois. Il a été réalisé progressivement, format par format en fonction des priorités du produit et des difficultés rencontrées.

Cette organisation permettait de travailler par étapes. Pour chaque format, il fallait comprendre les données disponibles, identifier les métadonnées utiles, développer le traitement adapté, puis vérifier les résultats obtenus.

Le travail a d'abord porté sur des formats qui étaient les plus utilisés par les clients de l'entreprise, comme les fichiers rasters et vectoriels. Il s'est ensuite poursuivi avec d'autres formats : GeoPackage, données CAO, Oracle, GeoParquet, SQL Server et, plus récemment, Nuage des points.

Cette progression correspondait aussi à la manière dont la migration devait être menée dans Scan. Chaque ajout devait rester compatible avec le fonctionnement existant du produit. Il ne s'agissait donc pas seulement de lire une donnée, mais de produire une sortie exploitable par les autres composants.

3.1.2 Périmètre fonctionnel selon les formats

Le périmètre n'était pas identique pour tous les formats. Certains ont fait l'objet d'un travail de documentation des métadonnées. D'autres ont nécessité des traitements complets, comme le listing ou la signature.

Pour les fichiers rasters, vectoriels et CAO, le travail réalisé concernait uniquement la documentation. L'objectif était d'extraire les informations techniques nécessaires à la création ou à la mise à jour des fiches de métadonnées dans Scan. Le listing et la signature n'ont pas été développés pour ces formats, car ils pouvaient être réalisés directement par le client de Scan.

Pour les formats suivants, le périmètre dépendait des besoins du produit, de la nature des données et des possibilités offertes par les bibliothèques utilisées. Cette approche progressive permettait d'avancer sans figer trop tôt une solution unique qui n'aurait pas forcément convenu à toutes les technologies.

Format	Travail réalisé ou en cours	Pris en charge par FME
Rasters, vecteurs et CAO	Documentation.	Oui
GeoPackage	Inventaire, documentation et signature.	Non
Oracle	Inventaire, documentation et signature.	Oui
GeoParquet	Documentation.	Non
SQL Server	Inventaire, documentation et signature.	Oui
Nuage des points	Travaux en cours sur l'inventaire, la documentation et la signature.	Oui

TABLE 3.1 : Progression des développements selon les formats étudiés

3.2 Développements sur les fichiers géographiques

3.2.1 Données rasters et vectorielles

Les premiers développements ont porté sur les données rasters et vectorielles. Ces formats constituaient un bon point d'entrée, car ce sont les plus utilisés par les clients de l'entreprise.

Pour les données vectorielles, deux formats ont été pris en compte dans un premier temps : le Shapefile et le GeoJSON. Ces formats sont très utilisés pour échanger des couches géographiques. Le but était de produire une description fiable de la ressource analysée, sans développer le listing ni la signature, qui pouvaient déjà être réalisés par le client de Scan pour ces formats.

Les informations recherchées étaient les mêmes pour le Shapefile et le GeoJSON. Elles concernaient d'abord l'identification de la ressource : son nom, son chemin, son format court et son format complet, ainsi que ses dates de création et de modification lorsqu'elles étaient disponibles. Le traitement devait aussi extraire le système de coordonnées, le nombre d'entités, le type de géométrie et l'enveloppe convexe de la couche.

La documentation devait également décrire la structure attributaire. Pour chaque champ, il fallait récupérer son nom, son type, sa taille et sa précision lorsque ces informations étaient présentes. Ces éléments permettaient de produire une fiche de métadonnées complète, à la fois sur la localisation de la donnée, sa géométrie et son contenu attributaire.

Pour les données rasters, plusieurs formats ont été pris en charge, notamment ECW, GeoTIFF, JP2, JPG, PNG et d'autres formats courants. La documentation portait sur l'identification du fichier, son format, son chemin, ses dates, son système de coordonnées et son enveloppe convexe. Elle devait aussi récupérer les dimensions de l'image, le nombre de bandes et, pour chaque bande, des informations comme le nom, l'interprétation et la profondeur.

GDAL et OGR ont été utilisés pour accéder à ces informations. Ces bibliothèques offrent une interface commune pour lire de nombreux formats. Elles permettent donc de limiter le nombre de traitements spécifiques à écrire pour chaque type de fichier.

Ce premier travail a permis de mettre en place une base de développement. Il a aussi permis d'identifier les premières différences de comportement entre les formats, notamment sur la manière dont les systèmes de coordonnées, les champs ou l'enveloppe convexe sont exposés.

3.2.2 GeoPackage

Le travail s'est ensuite poursuivi avec le format GeoPackage. Il s'agit d'un format de fichier permettant de stocker plusieurs couches géographiques dans un même fichier. Il est souvent utilisé pour échanger des données tout en conservant une structure proche d'une petite base de données.

La particularité du GeoPackage est qu'un même fichier peut contenir plusieurs types de données. Il peut regrouper des couches vectorielles, des rasters ou encore des tables sans géométrie. Le script devait donc d'abord analyser le contenu du fichier, identifier les couches présentes et déterminer leur type avant d'appliquer le traitement adapté.

Le développement a porté sur les trois traitements attendus : l'inventaire, la documentation et la signature. L'inventaire permettait de lister les couches présentes dans le fichier et de préciser, pour chacune, s'il s'agissait d'une couche vectorielle, d'un raster ou d'une table. La documentation produisait ensuite les métadonnées propres à chaque couche. La signature consistait à calculer une empreinte de la donnée afin de pouvoir suivre son évolution entre deux analyses.

Pour une couche raster contenue dans un GeoPackage, les informations recherchées lors de la documentation étaient par exemple le nom de la couche, le format, le chemin du fichier, les dates disponibles, le système de coordonnées, l'enveloppe convexe, le nombre de colonnes et de lignes, ainsi que le nombre de bandes. Pour chaque bande, le traitement devait aussi récupérer des informations comme le nom, l'interprétation et la profondeur.

Cette étape demandait donc une logique plus adaptable qu'un simple traitement de fichier. Le script ne pouvait pas appliquer une seule méthode à tout le contenu du GeoPackage. Il devait reconnaître la nature de chaque couche, puis produire une sortie cohérente avec ce que Scan attendait pour ce type de donnée.

3.2.3 GeoParquet

GeoParquet s'appuie sur le format Parquet, souvent utilisé pour stocker des données de manière efficace, notamment dans des contextes d'analyse.

Ce format n'était pas initialement pris en charge par les traitements FME existants. Il était aussi moins connu de l'équipe Produit, contrairement au GeoPackage dont les usages étaient déjà

mieux identifiés. Il n'y avait donc pas de résultat historique directement disponible pour servir de référence. Un premier travail d'étude a été nécessaire afin de comprendre la structure du fichier et la manière dont les informations géographiques y étaient enregistrées.

L'objectif était ensuite d'identifier les informations disponibles dans ce format et de voir comment les exploiter pour produire les métadonnées attendues par Scan. Le traitement devait notamment retrouver les informations géographiques, les champs et les éléments nécessaires à la description de la ressource.

Pour ce format, le travail a porté sur la documentation. Il fallait comprendre comment les informations spatiales étaient stockées dans le fichier et comment retrouver les éléments utiles : champs, géométrie, système de coordonnées et l'enveloppe convexe. Ce format a nécessité une phase d'expérimentation, car il ne se présente pas de la même manière qu'un fichier SIG traditionnel.

Les métadonnées recherchées étaient proches de celles des formats vectoriels : nom de la ressource, format, chemin du fichier, dates, nombre d'entités, système de coordonnées, type de géométrie, champs attributaires et enveloppe convexe. Le traitement devait aussi identifier le type de donnée manipulé afin de distinguer clairement le cas vectoriel d'autres usages possibles du format Parquet.

La prise en charge de GeoParquet a donc permis d'élargir le périmètre de la migration vers des formats plus modernes, tout en conservant la même exigence : produire des résultats exploitables par le Scan.

3.2.4 Données CAO

Une autre étape a concerné les données CAO. Ces données proviennent du dessin assisté par ordinateur. La difficulté principale venait du fait qu'il ne s'agissait pas toujours de données géographiques classiques. Un fichier CAO peut contenir des objets de dessin, des annotations ou des éléments textuels qui sont eux aussi représentés sous forme géométrique. Ces éléments peuvent alors être pris en compte dans l'emprise, même lorsqu'ils n'ont pas de signification géographique réelle.

Cette différence rendait l'implémentation plus délicate. Le traitement devait extraire les informations utiles à Scan, tout en évitant d'interpréter trop rapidement un dessin CAO comme une couche SIG classique.

Le travail sur ces formats a d'abord été engagé avec GDAL et OGR, comme pour les premiers formats étudiés. Cette piste semblait cohérente, car ces bibliothèques permettaient déjà de lire plusieurs types de données géographiques.

Cependant, cette approche a rapidement montré ses limites. Le format DWG, le plus utilisé des clients d'ISOGEO, n'était pas pris en charge par GDAL et OGR. Il a donc fallu rechercher une autre solution pour pouvoir exploiter ces données dans le cadre de Scan.

Après cette phase d'étude, la solution retenue a été *LibreDWG*. Il s'agit d'une bibliothèque libre spécialisée dans la lecture du format DWG, couramment utilisé par les logiciels de dessin assisté par ordinateur. Dans le cadre de Scan, elle a servi à accéder au contenu des dessins et à récupérer les informations exploitables pour la documentation, comme les couches, les objets et les types de géométrie. Cette étape était nécessaire, car les fichiers CAO ne présentent pas toujours les données de la même manière qu'un format SIG classique.

Les formats DWG et DXF ont été abordés avec cette logique. La documentation devait identifier le fichier, son format, son chemin, le nombre total d'objets et le type de géométrie. Elle devait aussi parcourir les couches internes du dessin afin de récupérer, pour chacune, son nom, son nombre d'objets et le type de géométrie associé.

Ces formats ne fournissent pas toujours les mêmes informations qu'une donnée SIG classique. Le format CAO est d'abord un format de dessin : il n'est donc pas forcément orienté géographie. Le calcul de l'enveloppe convexe n'était réalisé que lorsqu'un fichier de projection associé, par exemple avec l'extension `.prj`, permettait d'interpréter correctement les coordonnées. Dans les autres cas, le traitement devait conserver l'absence d'information au lieu de produire une valeur approximative.

Certaines limitations dépendaient aussi des technologies disponibles. Les formats SDF et DGN ont été écartés dans cette première phase de développement, car ils n'étaient pas supportés par GDAL/OGR ni par les bibliothèques open source étudiées. Ils étaient également moins utilisés par les clients. Le choix a donc été fait de concentrer les efforts sur les formats pouvant être traités de manière plus fiable.

Pour les données CAO, le travail est donc resté centré sur la documentation. L'objectif était d'extraire les informations réellement disponibles et utiles à Scan, sans chercher à forcer une interprétation SIG complète lorsque le format ne s'y prêtait pas. Cette étape a montré qu'une migration ne consiste pas toujours à tout reprendre immédiatement. Dans certains cas, il est préférable d'identifier clairement les limites, de documenter les raisons du blocage et de concentrer les efforts sur les formats pouvant être traités de manière fiable.

3.3 Développements sur les bases de données

Pour les bases de données, le choix a été différent de celui retenu pour les formats fichiers. Nous avons privilégié l'utilisation de bibliothèques natives, basées sur l'exécution de requêtes SQL. Cette approche permettait de s'appuyer sur les capacités propres des bases de données et de bénéficier de leurs performances, notamment lorsque les volumes à traiter étaient importants.

3.3.1 Oracle

Après les premiers travaux sur les fichiers, le développement s'est poursuivi sur Oracle. Cette technologie introduisait des contraintes différentes, car les données sont stockées dans une base

relationnelle, avec des paramètres de connexion, des schémas, des tables et des droits d'accès.

Le traitement devait donc gérer la connexion à la base, identifier les tables accessibles et récupérer les informations utiles à la documentation. Il fallait aussi tenir compte des particularités d'Oracle Spatial pour les données géographiques.

Pour Oracle, la bibliothèque *python-oracledb* a été utilisée. Elle permettait d'exécuter directement les requêtes nécessaires pour l'inventaire, la documentation et la signature. Cette solution évitait de rapatrier inutilement les données côté Python lorsque la base pouvait réaliser elle-même certaines opérations.

Les tests ont pu être réalisés à partir d'un jeu de données disponible sur un serveur de l'entreprise. Cela permettait de travailler sur des tables déjà utilisées par les workbenches FME et de vérifier les résultats sur des données représentatives.

Pour la documentation, les informations recherchées concernaient principalement la table analysée, son schéma, son emplacement dans la base, le nombre d'entités, le type de géométrie, le système de coordonnées et l'enveloppe convexe. Le traitement devait aussi décrire les champs de la table, avec leur nom, leur type et leur alias lorsqu'il était disponible. Ces éléments permettaient de produire une fiche cohérente avec celles générées pour les formats fichiers.

Cette étape a demandé une attention particulière sur la robustesse. Une base de données peut être inaccessible, une table peut ne pas contenir de géométrie ou un utilisateur peut ne pas avoir les droits nécessaires. Le traitement devait donc produire un comportement clair dans ces situations.

Le travail sur Oracle a renforcé l'idée que chaque format ou technologie possède ses propres contraintes. Même lorsque les métadonnées recherchées sont proches, la manière de les obtenir peut changer fortement selon la technologie.

3.3.2 SQL Server

Le développement a également concerné Microsoft SQL Server. Comme pour Oracle, il s'agit d'une base relationnelle pouvant contenir des données spatiales.

La prise en charge de cette technologie impliquait de gérer les paramètres de connexion, les bases, les schémas et les tables accessibles. Il fallait aussi tenir compte des types géographiques disponibles dans SQL Server et de la façon dont les informations spatiales peuvent être interrogées.

Pour SQL Server, le choix s'est porté sur *SQLAlchemy*. Cette bibliothèque permettait de structurer les accès à la base tout en conservant une logique reposant sur des requêtes SQL. Comme pour Oracle, l'objectif était de profiter des capacités de la base pour limiter les traitements inutiles côté application.

La mise en place des tests a demandé un travail supplémentaire. Contrairement à Oracle, l'entreprise ne disposait pas d'un jeu de données SQL Server directement exploitable pour cette

tâche. De plus, l'espace disponible sur le serveur de test était saturé. Un environnement local a donc été créé pour disposer d'une base SQL Server de référence.

Cette base locale a été alimentée avec un échantillon de tables migrées depuis une base PostgreSQL/PostGIS existante. L'objectif était de conserver un volume et une diversité de données suffisants pour tester les scripts dans des conditions proches d'un usage réel. Après la migration, plusieurs contrôles ont été réalisés afin de vérifier le nombre d'entités, les types de champs et la validité des géométries.

Une fois cet environnement de test validé, les traitements ont été adaptés aux spécificités de SQL Server, tout en conservant une sortie cohérente avec les autres formats et technologies. Les métadonnées attendues portaient sur le nom de la table spatiale, le chemin de connexion, le nombre d'entités, le type de géométrie, le système de coordonnées, l'enveloppe convexe et la structure des attributs.

L'objectif restait le même : permettre à Scan de recenser, documenter et signer les ressources sans dépendre des anciens traitements.

3.4 Traitement des nuages de points

Les travaux les plus récents portent sur les nuages de points, en particulier les fichiers LAS et LAZ. Le format LAS est utilisé pour stocker des points issus de relevés comme le LiDAR. Chaque point peut porter des informations comme ses coordonnées, son altitude ou d'autres valeurs mesurées. Le format LAZ correspond à une version compressée de LAS, plus légère à stocker. Ces données sont particulières, car elles peuvent représenter des volumes très importants et ne se manipulent pas comme une couche vectorielle classique.

Comme pour les premiers formats étudiés, une première piste consistait à regarder ce que GDAL et OGR permettaient de faire. Cette approche était logique, car ces bibliothèques étaient déjà utilisées dans plusieurs traitements de Scan. Cependant, les nuages de points ont rapidement montré des besoins plus spécifiques, notamment sur la lecture des fichiers.

L'objectif est donc d'identifier les métadonnées pertinentes pour décrire ces ressources dans Scan, sans parcourir inutilement l'ensemble des points lorsque ce n'est pas nécessaire. Il faut notamment comprendre les informations disponibles dans les fichiers, les formats pris en charge et les limites liées aux très gros volumes.

L'étude des alternatives porte principalement sur deux bibliothèques : PDAL et Laspy. PDAL est spécialisée dans le traitement des nuages de points. Elle est adaptée aux traitements géospatiaux et permet de lire différents formats de ce type de donnée. Laspy est une bibliothèque Python plus ciblée sur la lecture et l'écriture des fichiers LAS et LAZ.

Cette partie est encore en cours. Le choix de la solution doit tenir compte de la richesse des métadonnées accessibles, de la performance sur de gros fichiers et de la capacité à intégrer proprement le traitement dans Scan.

3.5 Gestion des erreurs et des connexions

Au cours des développements, certains aspects transverses étaient aussi importants que le traitement des formats eux-mêmes. C'était notamment le cas de la gestion des erreurs et des modes de connexion aux bases de données.

3.5.1 Gestion des erreurs

Les scripts devaient signaler clairement les erreurs rencontrées pendant les traitements. L'objectif était d'éviter les échecs silencieux ou les messages trop génériques, difficiles à exploiter ensuite.

Cette remontée des erreurs est importante pour le support. Lorsqu'un traitement échoue chez un client, le message retourné doit aider à comprendre rapidement la cause du problème : paramètre manquant, connexion impossible, table absente, donnée invalide, géométrie non lisible ou source vide. Un message précis permet de mieux orienter le diagnostic et de réduire le temps nécessaire pour corriger ou expliquer l'erreur.

La gestion des erreurs devait donc rester cohérente avec le fonctionnement de Scan. Le traitement devait indiquer ce qui n'avait pas pu être réalisé, sans masquer l'information utile et sans interrompre inutilement l'analyse des autres ressources lorsque cela était possible.

3.5.2 Modes de connexion aux bases de données

Les bases de données demandaient également une attention particulière sur la connexion. Les utilisateurs ne se connectent pas toujours de la même manière selon leur environnement, leur base et les règles de sécurité en place.

Pour Oracle, le traitement devait gérer plusieurs cas. La connexion pouvait se faire à partir d'un alias ou d'une chaîne TNS (un identifiant Oracle qui référence les paramètres de connexion à une base). Elle pouvait aussi être construite à partir des informations classiques de connexion : serveur, port et nom de base ou de service. Ces deux modes devaient être pris en charge afin de rester compatible avec les configurations déjà utilisées par les clients.

Pour SQL Server, deux grands cas de connexion étaient à gérer. Lorsque l'utilisateur fournissait un identifiant et un mot de passe, le traitement utilisait une authentification SQL Server. Lorsque ces informations n'étaient pas fournies, la connexion pouvait s'appuyer sur l'authentification Windows de la machine. Le script devait aussi tenir compte du pilote disponible sur le poste et des paramètres nécessaires pour établir la connexion.

Cette prise en charge des différents modes de connexion était indispensable pour que les scripts puissent fonctionner dans des environnements variés, sans imposer une seule manière de se connecter aux bases.

3.6 Principe de signature des données

La signature permet d'associer une empreinte à une donnée. Cette empreinte est calculée à partir des informations qui décrivent son contenu. Si la donnée change, la signature doit changer également. Elle sert donc à vérifier l'intégrité d'une ressource et à repérer les évolutions entre deux analyses.

L'objectif n'était pas seulement de produire un identifiant. La signature devait prendre en compte les éléments réellement significatifs : la structure de la donnée, ses attributs, ses géométries lorsqu'elles existaient et certaines informations spatiales comme le système de coordonnées.

3.6.1 Signature des fichiers GeoPackage

Pour GeoPackage, la signature était calculée couche par couche. Le traitement commençait par identifier les champs à prendre en compte, puis parcourait les entités de la couche. Pour chaque entité, une première empreinte était produite à partir des valeurs attributaires. Lorsque la couche contenait des géométries, la géométrie de l'entité était également intégrée au calcul.

Une attention particulière était portée aux géométries absentes. Lorsqu'une entité ne possédait pas de géométrie, cette absence était explicitement prise en compte dans la signature. Cela évitait de confondre une entité sans géométrie avec une entité dont la géométrie aurait simplement été ignorée.

Les empreintes calculées pour chaque entité étaient ensuite triées avant de produire la signature finale de la couche. Ce tri permettait de ne pas dépendre de l'ordre de lecture des entités dans le fichier. Ainsi, deux couches ayant le même contenu devaient produire la même signature, même si les entités n'étaient pas lues dans le même ordre.

3.6.2 Signature des bases de données

Pour Oracle et SQL Server, le principe restait le même, mais le traitement s'appuyait sur des requêtes SQL. La signature devait tenir compte du contenu de la table, de sa structure et, lorsqu'elle existait, de sa dimension spatiale.

Le traitement commençait par compter les entités de la table. Si la table était vide, aucune signature de contenu n'était calculée et une erreur spécifique était retournée. Ensuite, le script vérifiait la présence d'une colonne géométrique. Lorsque des géométries étaient présentes, le système de coordonnées était intégré à la signature afin que la projection fasse partie de l'empreinte de la donnée.

Les champs non géométriques étaient ensuite récupérés et ordonnés. Cet ordre était important : il permettait de signer la structure de la table de manière stable. Une modification de schéma, comme l'ajout ou la suppression d'une colonne, devait donc être visible dans la signature.

Le contenu de la table était ensuite lu par lots afin de pouvoir traiter des volumes importants sans charger toute la donnée en mémoire. Chaque ligne était normalisée, puis ajoutée au calcul de l’empreinte. Cette méthode permettait de prendre en compte les valeurs attributaires, les géométries et la structure générale de la table, tout en conservant un traitement adapté aux gros volumes.

3.7 Intégration, revue de code et exécutable

3.7.1 Revue de code et intégration progressive

Après le développement d’un traitement, le code était soumis à une *Pull Request*. Cette étape fait partie des règles de développement de l’équipe Produit. Elle permettait de proposer les modifications, de les relire et de les discuter avant leur mise en production.

La revue de code était réalisée par mon tuteur d’alternance. Son objectif était de vérifier plusieurs points : la cohérence avec l’architecture existante, la lisibilité du code, la gestion des erreurs, la performance, la maintenabilité et le respect des formats de sortie attendus par Scan. Elle permettait aussi de repérer des cas limites qui n’avaient pas forcément été identifiés pendant le développement.

Les retours étaient structurés selon trois niveaux. Un retour **critique** signalait un problème à corriger avant intégration, par exemple un comportement incorrect, un risque de régression ou une erreur pouvant bloquer l’utilisation du traitement. Un **avertissement** indiquait un point important à revoir ou à justifier, sans bloquer nécessairement toute la fonctionnalité. Une **suggestion** proposait une amélioration de lisibilité, de structure ou de style.

Une fois les retours reçus, je devais corriger le code lorsque cela était nécessaire ou expliquer les choix effectués. Les scripts étaient ensuite soumis à une nouvelle revue. Cette boucle se poursuivait jusqu’à ce que les points critiques soient levés et que le code soit considéré comme suffisamment stable pour être intégré.

3.7.2 Suivi de l’exécutable au fil des développements

La génération de l’exécutable n’a pas été traitée comme une étape finale indépendante. Une fois l’image Docker de construction mise en place, elle a été réutilisée au fur et à mesure de l’avancement des scripts.

Docker restait ainsi le seul prérequis pour construire l’exécutable Windows. L’environnement de construction contenait les versions nécessaires de Python, PyInstaller, GDAL et des autres bibliothèques utilisées par Scan. Cette organisation évitait de dépendre des installations présentes sur le poste du développeur.

Après l’ajout ou la modification de certains traitements, l’exécutable pouvait être régénéré

pour vérifier que les scripts s'intégraient toujours correctement. Les contrôles portaient sur le lancement du programme, la présence des dépendances nécessaires et la cohérence des résultats produits.

Une attention particulière était portée aux bibliothèques géospatiales, car elles reposent sur des dépendances natives. Le packaging devait donc embarquer les éléments nécessaires afin que Scan puisse être exécuté sans installation locale de Python ou de bibliothèques géographiques.

Cette vérification régulière permettait de garder un lien direct entre les développements réalisés et leur utilisation réelle dans Scan. Elle limitait le risque de découvrir trop tard un problème lié au packaging ou à une dépendance.

CHAPITRE 4

Résultats, validation et bilan

Ce chapitre présente les résultats obtenus. Il ne reprend pas le détail de chaque développement, déjà présenté dans le chapitre précédent. Il met plutôt en évidence la manière dont les traitements ont été validés, les formats couverts, les limites rencontrées et les apports de la mission.

4.1 Résultats obtenus

Les résultats peuvent être présentés selon les trois traitements principaux de Scan : le *listing*, le *lookup* et la signature.

Les captures d'écran insérées dans cette section permettent d'illustrer les sorties produites pour quelques formats représentatifs. Elles montrent à la fois la structure des résultats et la manière dont les informations sont retournées à Scan.

Traitement	Résultat attendu	Exemples de formats concernés
Listing	Recenser les ressources accessibles dans un fichier ou une base de données.	GeoPackage, Oracle, SQL Server.
Lookup	Extraire les métadonnées nécessaires à la création ou à la mise à jour des fiches.	Rasters, vecteurs, GeoPackage, GeoParquet, CAO, Oracle, SQL Server.
Signature	Produire une empreinte permettant de détecter les changements sur une donnée.	GeoPackage, Oracle, SQL Server.

TABLE 4.1 : Synthèse des résultats selon les traitements de Scan

4.1.1 Résultats du listing

Le listing correspond à l'inventaire des ressources accessibles. Il permet à Scan d'identifier ce qui peut ensuite être documenté ou signé. Selon le format, le résultat peut correspondre à une liste de couches, de tables ou de ressources présentes dans une connexion.

Pour GeoPackage, le listing devait parcourir le fichier et identifier les couches disponibles.

Pour les bases Oracle et SQL Server, le listing devait récupérer toutes les tables auxquelles

l'utilisateur a accès selon les paramètres de connexion utilisés.

```
Users > MarcelAssie > Documents > isogeo > Output > FILES > Geopackage > listing_geopackage_result.json > ...  
[  
  {  
    "result": {  
      "featureType": "small_world_and_byte",  
      "dataType": "raster",  
      "path": "C:\\Users\\MarcelAssie\\Documents\\isogeo\\Data\\DataTests\\FILES\\GPKG\\small_world_and_byte.gpkg"  
    }  
  },  
  {  
    "result": {  
      "featureType": "utm",  
      "dataType": "raster",  
      "path": "C:\\Users\\MarcelAssie\\Documents\\isogeo\\Data\\DataTests\\FILES\\GPKG\\small_world_and_byte.gpkg"  
    }  
  }  
]
```

FIGURE 4.1 : Exemple de résultat de listing pour un fichier GeoPackage

```
[  
  {  
    "result": {  
      "featureType": "demo.marches_decouverts"  
    }  
  },  
  {  
    "result": {  
      "featureType": "demo.pav_colonne_verre"  
    }  
  },  
  {  
    "result": {  
      "featureType": "demo.pav_composteurs"  
    }  
  },  
  {  
    "result": {  
      "featureType": "demo.pav_stations_trilib"  
    }  
  },  
  {  
    "result": {  
      "featureType": "demo.que_faire_en_RI"  
    }  
  }  
]
```

FIGURE 4.2 : Exemple de résultat de listing pour une base de données SQL Server

4.1.2 Résultats du lookup

Les résultats obtenus varient selon la nature de la donnée. Pour les formats vectoriels, le lookup permet de récupérer le nom, le format, le chemin, le système de coordonnées, le type de géométrie, le nombre d'entités, les champs et l'enveloppe convexe. Pour les rasters, il permet

aussi de récupérer les dimensions de l'image, le nombre de bandes et les propriétés associées aux bandes.

Le lookup a également été adapté à des formats plus particuliers. Pour GeoPackage, le traitement devait s'adapter au type de couche rencontré. Pour GeoParquet. Pour les données CAO, on a lister toutes les couches du fichier devaient figurées dans les métadonnées.

```
[
  {
    "dataset": {
      "name": "Une belle ligne",
      "formatShort": "SHAPEFILE",
      "formatLong": "ESRI Shapefile",
      "created": "Tue Dec 23 12:24:45 2025",
      "modified": "Mon Jun 27 11:57:03 2022",
      "numberOfFeatures": 1,
      "coordsys": {
        "name": "RGF93 v1 / Lambert-93",
        "EPSG": "2154"
      },
      "type": "LINESTRING",
      "path": "C:\\Users\\MarcelAssie\\Documents\\isogeo\\Data\\DataTests\\FILES\\SHP\\mojito\\Une belle ligne.shp",
      "attributes": [
        {
          "name": "id",
          "type": "Real",
          "width": 20,
          "precision": 0
        },
        {
          "name": "1 champ",
          "type": "Real",
          "width": 20,
          "precision": 0
        },
        {
          "name": "2 champs",
          "type": "Real",
          "width": 20,
          "precision": 0
        }
      ],
      "envelope": "{\"type\": \"LineString\", \"coordinates\": [[2.3334532, 48.8263593], [1.5287712, 45.1426394]]}",
      "gdal_version": "3.12.1"
    }
  }
]
```

FIGURE 4.3 : Exemple de résultat de lookup pour un fichier Shapefile

```

C: > Users > MarcelAssie > Documents > isogeo > Output > FILES > CAO > true > {} d_CAO_Dijon_2007_dwg.json > ...
1  [
2  {
3      "dataset": {
4          "name": "Dijon_2007",
5          "layers": [
6              {
7                  "name": "0",
8                  "numberOfFeatures": 1,
9                  "type": "no_geom"
10             },
11             {
12                 "name": "GIS_CART_EV_PUBLIC",
13                 "numberOfFeatures": 600,
14                 "type": "LINESTRING"
15             },
16             {
17                 "name": "GIS_VOIE_SURFACIQUE_RUES",
18                 "numberOfFeatures": 3831,
19                 "type": "LINESTRING"
20             },
21             {
22                 "name": "GIS_VOIE_HAB_VOIRIE_SURF",
23                 "numberOfFeatures": 2274,
24                 "type": "LINESTRING"
25             },
26             {
27                 "name": "GIS_COM_CONSEIL_QUARTIER",
28                 "numberOfFeatures": 1971,
29                 "type": "LINESTRING"
30             },
31             {
32                 "name": "GIS_BATI",
33                 "numberOfFeatures": 23849,
34                 "type": "LINESTRING"
35             },
36             {
37                 "name": "GIS_CART_BATI_MAJ",
38                 "numberOfFeatures": 414,
39                 "type": "LINESTRING"
40             }
          ]
        }
      }
    ]
  
```

(a) Première partie du lookup d'un fichier DWG

```

    {
      "name": "ICSP_STATIONS_ICSP",
      "numberOfFeatures": 30,
      "type": "POINT"
    },
    {
      "name": "COMMUNES",
      "numberOfFeatures": 581,
      "type": "LINESTRING"
    },
    {
      "name": "GIS_COMMUNES",
      "numberOfFeatures": 5337,
      "type": "LINESTRING"
    }
  ],
  "env": {
    "gdalVersion": "3.12.4",
    "libreDWGVersion": "0.13.4"
  },
  "formatShort": "dwg",
  "formatLong": "AutoCAD DWG",
  "numberOfFeatures": 164660,
  "coordsys": {
    "name": "RGF93 v1 / CC47",
    "EPSG": "3947"
  },
  "type": "GEOMETRYCOLLECTION",
  "path": "C:\\Users\\MarcelAssie\\Documents\\isogeo\\Data\\DataTests\\FILES\\CAO\\DWG\\Dijon_2007.dwg",
  "envelope": "{ \"type\": \"Polygon\", \"coordinates\": [[ [ [ 15.0900428, 47.2862466 ], [ 4.981815, 47.3021446 ], [ 4.981833, 47.3021674 ], [ 4.9817691, 47.3022014 ], [ 15.0900428, 47.2862466 ] ] ] ] }"
}
  
```

(b) Suite du lookup du même fichier DWG

FIGURE 4.4 : Exemple de résultat de lookup pour une donnée CAO au format DWG

4.1.3 Résultats de la signature

Le résultat de sortie d'une signature n'est pas une métadonnée descriptive, mais une valeur de contrôle. Cette valeur doit rester stable si la donnée ne change pas et évoluer lorsqu'une modification significative est apportée.

Les captures retenues permettent de montrer trois situations différentes. Pour une donnée géographique, la clé `is_geo` prend la valeur `1`. Pour une donnée non géographique, elle prend la valeur `0`. Enfin, lorsqu'une donnée ne contient aucune entité, la clé `is_geo` reste à `null` et une erreur est remontée. Ces cas sont importants, car ils ne produisent pas exactement le même type de résultat.

```
[
  {
    "feature": {
      "signatures": "f6baed5c0c7529b7fc20dc2304953875",
      "is_geo": 1,
      "error": null,
      "env": {
        "oracledbVersion": "3.4.2"
      }
    }
  }
]
```

(a) Signature d'une donnée géographique

```
[
  {
    "feature": {
      "signatures": "6751c17c3024974d10093d238f0b247c",
      "is_geo": 0,
      "error": null,
      "env": {
        "oracledbVersion": "3.4.2"
      }
    }
  }
]
```

(b) Signature d'une donnée non géographique

```
[
  {
    "feature": {
      "signatures": "cae66941d9efbd404e4d88758ea67670",
      "is_geo": null,
      "error": "nothingToSign",
      "env": {
        "oracledbVersion": "3.4.2"
      }
    }
  }
]
```

(c) Signature d'une donnée sans entité

FIGURE 4.5 : Exemples de résultats de signature selon le type de donnée

4.2 Validation des traitements développés

La validation occupait une place importante dans la mission. La migration ne consistait pas seulement à réécrire des traitements en Python. Il fallait aussi montrer que les résultats produits étaient fiables, cohérents et exploitables par Scan.

Pour rappel, trois familles de tests ont été utilisées : les tests de non-régression, les tests d'intégrité de la signature et les tests de performance.

Type de test	Objectif	Traitements concernés
Non-régression	Comparer les sorties Python avec les résultats de référence produits par les anciens traitements.	Inventaire et documentation.
Intégrité de la signature	Vérifier que l’empreinte calculée réagit correctement aux modifications de la donnée.	Signature.
Performance	Mesurer le comportement des scripts sur des volumes de données importants.	Inventaire, documentation et signature.

TABLE 4.2 : Principales familles de tests utilisées pour valider les traitements

4.2.1 Tests de non-régression

Les tests de non-régression concernaient principalement le listing (recensement) et le lookup (la documentation). Ils consistaient à comparer les résultats produits par les nouveaux scripts Python avec les résultats issus des traitements FME de référence.

La comparaison portait sur les informations attendues par Scan : nom de la ressource, format (court comme long), chemin vers la source de données, système de coordonnées, type de géométrie, nombre d’entités, enveloppe convexe, champs attributaires ou encore propriétés propres aux rasters. L’objectif n’était pas seulement d’obtenir un fichier de sortie valide, mais de vérifier que les métadonnées restaient cohérentes avec le comportement historique du produit.

Lorsque des écarts étaient observés, ils étaient analysés au cas par cas. Certains écarts révélaient une erreur à corriger dans le script Python. D’autres pouvaient s’expliquer par une différence normale entre les bibliothèques utilisées ou par une limite du traitement de référence. Dans ce cas, l’écart devait être compris et documenté.

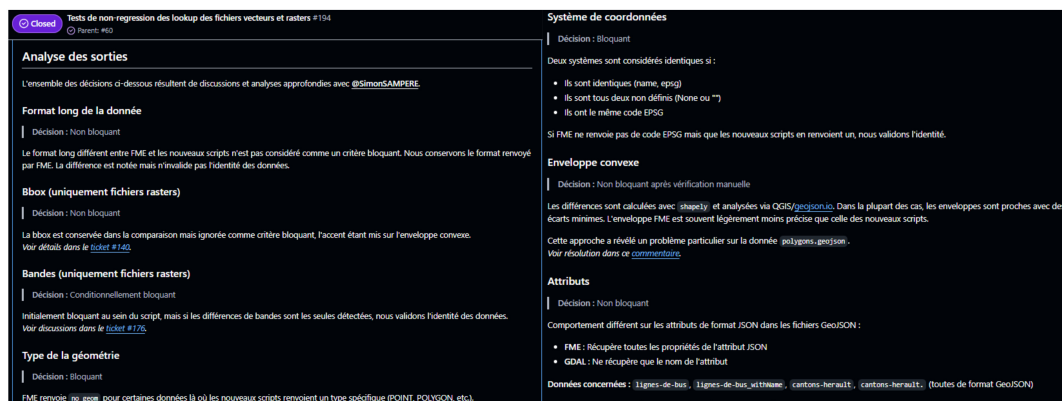


FIGURE 4.7 : Méthode de décision utilisée pour valider les tests de non-régression

4.2.2 Tests d'intégrité de la signature

Les tests d'intégrité concernaient la signature. L'objectif était de vérifier que l'empreinte produite restait stable lorsque la donnée ne changeait pas, mais qu'elle évoluait dès qu'une modification significative était appliquée.

L'image suivante présente un exemple de résultats obtenus. Pour chaque donnée, plusieurs scénarios sont exécutés : constance de la signature, modification attributaire, modification géométrique, changement de structure, changement spatial ou absence de la donnée. Le résultat attendu est que chaque scénario soit correctement détecté ou rejeté selon le cas testé.

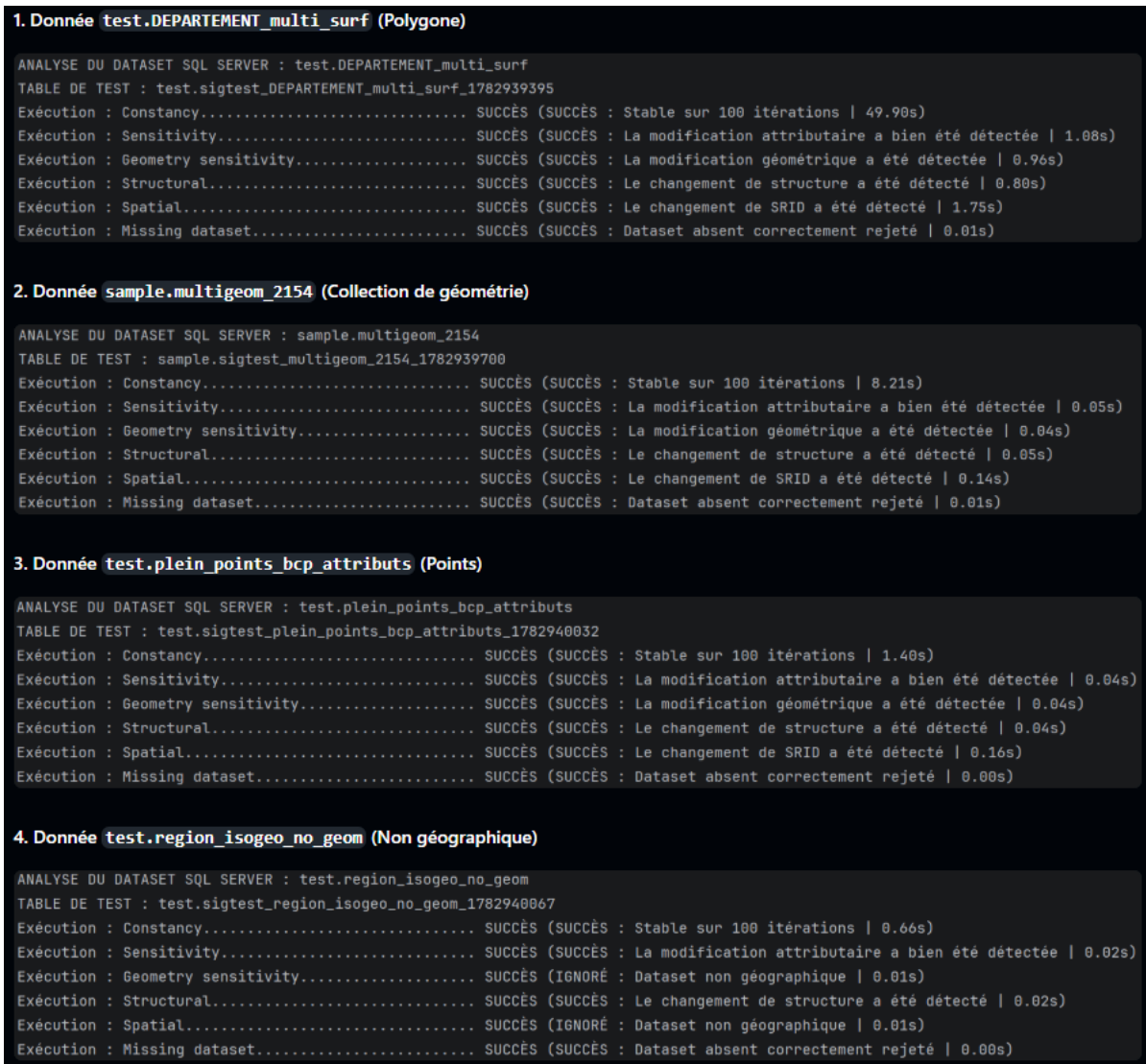


FIGURE 4.8 : Exemple de résultats des tests d'intégrité de la signature

Les résultats visibles indiquent que les scénarios testés aboutissent au comportement attendu. La constance montre que la signature reste stable sur plusieurs exécutions lorsque la donnée n'est pas modifiée. Les tests de sensibilité confirment ensuite que les modifications attributaires, géométriques, structurelles ou spatiales sont bien détectées.

L'image montre aussi que les cas particuliers sont pris en compte. Une donnée non géographique ne déclenche pas de test géométrique, car ce contrôle n'a pas de sens dans ce contexte. De même, lorsqu'une donnée est absente, le traitement doit la rejeter correctement au lieu de produire une signature trompeuse. Ces résultats permettent donc de vérifier que la signature est stable, sensible aux bons changements et cohérente avec la nature de la donnée analysée.

4.2.3 Tests de performance

Les tests de performance visaient à vérifier le comportement des scripts sur des volumes de données importants. Ils concernaient l'inventaire, la documentation et la signature.

Ces tests étaient nécessaires car Scan peut être utilisé sur des environnements contenant de nombreuses tables, des fichiers volumineux ou des couches avec un grand nombre d'entités. Un traitement fonctionnel sur un petit jeu de données peut devenir trop lent ou trop coûteux lorsqu'il est appliqué à un volume plus important.

Les mesures portaient principalement sur les temps d'exécution et sur les opérations les plus sensibles. Pour la signature, par exemple, la lecture par lots permettait de traiter de grandes tables sans charger tout le contenu en mémoire. Pour la documentation, l'enjeu était de récupérer les métadonnées utiles sans effectuer des calculs inutilement coûteux.

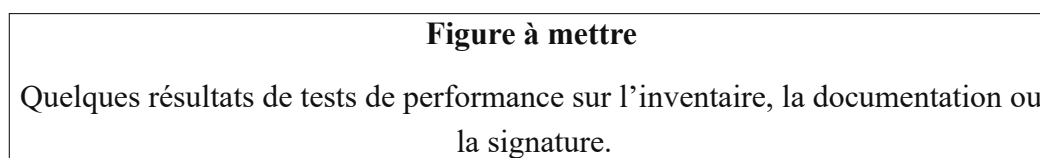


FIGURE 4.9 : Exemples de résultats de tests de performance

4.3 Limites rencontrées

La mission a aussi mis en évidence plusieurs limites. Certaines étaient liées aux formats eux-mêmes, d'autres aux environnements de test ou au packaging.

4.3.1 Limites liées aux formats

Tous les formats ne présentaient pas le même niveau de prise en charge. Les données CAO en sont un bon exemple. Les formats DWG et DXF proviennent du dessin assisté par ordinateur : ils peuvent contenir des objets graphiques, du texte ou des éléments de dessin qui ne correspondent pas toujours à une donnée SIG classique.

La première difficulté a donc été de comprendre les informations réellement disponibles dans ces fichiers et la manière de les récupérer sans leur donner un sens géographique qu'elles n'avaient pas forcément. Ce travail d'étude était nécessaire pour produire des métadonnées fiables et cohérentes avec ce que le fichier contenait réellement.

Les formats SDF et DGN ont été écartés dans cette première phase. Leur prise en charge par les bibliothèques étudiées n'était pas suffisamment fiable et ils étaient moins utilisés par les clients. Ce choix a permis de concentrer les efforts sur les formats pouvant être traités de manière plus stable.

4.3.2 Limites liées aux environnements de test

La disponibilité des jeux de données variait selon les technologies. Pour Oracle, des données de test étaient disponibles sur un serveur de l'entreprise. Pour SQL Server, il a fallu créer un environnement local afin de disposer d'un jeu de données représentatif.

Cette situation montre qu'une migration ne dépend pas seulement du code. Elle nécessite aussi des données fiables pour vérifier les résultats. Sans jeux de tests suffisamment variés, il devient plus difficile d'identifier les écarts, de tester les cas limites et de mesurer correctement les performances.

4.3.3 Limites liées au packaging

Le packaging sous Windows reste une partie sensible du projet. Les bibliothèques géospatiales reposent sur des dépendances natives et sur des données internes, comme celles utilisées par PROJ. Elles doivent être embarquées correctement avec l'exécutable pour éviter de dépendre de l'environnement local de la machine.

La mise en place de Docker a permis de rendre la génération plus reproductible, mais cette étape demande une attention continue. Les versions des bibliothèques, les chemins internes et les fichiers embarqués doivent rester cohérents pour que l'exécutable fonctionne de manière stable.

4.4 Apports de l'alternance

Cette mission m'a permis de progresser sur plusieurs dimensions : technique, méthodologique et professionnelle.

4.4.1 Apports techniques

Sur le plan technique, l'alternance m'a permis de renforcer ma pratique de Python dans un contexte concret de géomatique. J'ai travaillé sur des formats et technologies variés : fichiers vectoriels, rasters, GeoPackage, GeoParquet, CAO, Oracle, SQL Server et nuages de points.

J'ai également approfondi l'utilisation de bibliothèques spécialisées comme GDAL/OGR, *python-oracledb* ou SQLAlchemy. Ces outils imposent de comprendre les formats manipulés, leurs limites et les informations qu'ils permettent réellement d'obtenir.

Cette alternance m'a aussi permis de renforcer ma pratique de Git dans un cadre professionnel. J'ai appris à organiser mon travail, à versionner correctement mes scripts, à soumettre des modifications sous forme de Pull Request et à prendre en compte les retours de revue de code.

Le travail réalisé m'a également sensibilisé à la qualité du code. Il ne s'agissait pas seulement de produire un script fonctionnel, mais aussi un code lisible, maintenable, cohérent avec l'existant

et suffisamment clair pour être repris par d'autres développeurs.

Enfin, le travail sur le packaging m'a permis de mieux comprendre les enjeux de distribution d'une application Python. Une solution doit pouvoir être exécutée dans un environnement réel, avec des dépendances maîtrisées et un processus de génération reproductible.

4.4.2 Apports méthodologiques

La mission m'a appris à aborder un existant complexe de manière progressive. Il fallait lire le code déjà présent, comprendre les choix réalisés avant mon arrivée, documenter ce qui avait été compris et avancer sans rompre le fonctionnement attendu par Scan.

La méthode de validation a aussi été un apport important. Les tests de non-régression, les tests d'intégrité et les tests de performance m'ont permis de mieux structurer mon travail. Ils donnaient un cadre pour vérifier les résultats et pour discuter les écarts avec l'équipe.

La revue de code a également joué un rôle central. Les retours de mon tuteur permettaient d'améliorer les scripts, de justifier certains choix et de progresser vers une solution plus robuste.

4.4.3 Apports professionnels

Sur le plan professionnel, cette alternance a d'abord été une expérience d'intégration réussie au sein de l'entreprise. L'humain y occupe une place importante et les valeurs de collaboration, d'écoute et de confiance sont réellement présentes dans le quotidien de travail.

Cette intégration s'est ensuite construite plus précisément au sein de l'équipe Produit. Les points de suivi, les stand-up meetings, les revues de sprint et les échanges techniques m'ont aidé à mieux comprendre l'organisation du travail en entreprise et la manière dont un produit logiciel évolue collectivement.

J'ai aussi gagné en autonomie. Les tâches confiées demandaient de chercher des solutions, de comparer plusieurs approches, de documenter les résultats et de présenter les choix effectués. Cette progression s'est faite avec un accompagnement régulier, mais aussi avec une responsabilité croissante dans la conduite des développements.

4.5 Perspectives et recommandations

Les travaux réalisés constituent une étape dans la migration progressive des traitements de Scan. Ils ouvrent plusieurs perspectives pour la suite du projet.

4.5.1 Poursuite de la migration

La première perspective consiste à poursuivre la migration sur les traitements encore en cours ou non couverts. Les nuages de points devront notamment être approfondis afin de choisir l'approche la plus adaptée entre les solutions étudiées.

La migration pourra également être étendue à d'autres formats ou technologies selon les besoins du produit et les usages des clients. Cette poursuite devra rester progressive, car chaque format apporte ses propres contraintes.

4.5.2 Enrichissement des métadonnées

Une autre perspective concerne l'enrichissement des métadonnées produites par Scan. Les traitements Python et les bibliothèques utilisées permettent déjà de récupérer de nombreuses informations sur les ressources analysées. La suite pourrait consister à exploiter davantage ces possibilités afin de produire des fiches plus complètes.

Une piste complémentaire serait d'étudier l'utilisation de modèles d'intelligence artificielle pour générer automatiquement des suggestions de descriptions pour certaines métadonnées. L'objectif ne serait pas de remplacer la validation humaine, mais d'aider au remplissage manuel des champs et de rendre le catalogage moins chronophage.

Cette approche devrait d'abord rester dans un cadre de recherche et développement. Elle nécessiterait de vérifier la qualité des suggestions, leur cohérence avec les données analysées et leur intérêt réel pour les utilisateurs de Scan.

4.6 Conclusion

Ce rapport a présenté le travail réalisé pendant mon alternance autour de la migration progressive des traitements de Scan vers une solution Python plus autonome. Les développements menés ont contribué à faire avancer cette migration sur plusieurs formats et technologies, avec des niveaux d'avancement différents selon les cas : documentation, inventaire, signature ou étude préparatoire.

La validation a constitué un point essentiel du travail. Les tests de non-régression ont permis de comparer les nouvelles sorties Python avec les références existantes lorsque cela était possible. Les tests d'intégrité ont servi à vérifier la fiabilité des signatures. Les tests de performance ont permis de contrôler le comportement des scripts sur des volumes plus importants.

La mission a aussi montré que la migration ne se résume pas à remplacer un outil par un autre. Elle demande de comprendre les formats, de gérer leurs limites, de préparer des environnements de test fiables et de s'assurer que les traitements restent compatibles avec les besoins de Scan.

Au-delà des résultats techniques, cette alternance m'a permis de progresser dans un contexte

professionnel exigeant, au contact d'une équipe Produit et de méthodes de développement structurées. Les perspectives restent ouvertes, notamment autour de la poursuite de la migration, du renforcement des tests et de la stabilisation du processus de génération de l'exécutable.